An augmented Lagrangian algorithm for constrained nonlinear least-squares

Pierre Borie*, Fabian Bastin*, Stéphane Dellacherie†‡

June 21, 2026

Abstract

We present an algorithm for solving nonlinear least-squares problems subject to a mix of nonlinear and linear constraints. The nonlinear constraints are handled by reformulating the objective as the augmented Lagrangian function while linear constraints are handled directly. Each iteration consists in approximately solving a linearly constrained problem by means of a gradient projection technique. Our approach also involves a structured approximation of the augmented Lagrangian Hessian. We show global convergence of the method and provide numerical experiments.

Keywords: constrained nonlinear least-squares, augmented Lagrangian method, Julia, structured quasi-Newton update.

1 Introduction

We consider the constrained nonlinear least-squares (NLS) problem

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & \frac{1}{2}\|r(x)\|^2 \\ \text{s.t.} \quad & c(x) = 0 \\ & x \in \Omega \end{aligned} \tag{1.1}$$

where $\|\cdot\|$ denotes the ℓ_2 norm. The residuals $r\colon \mathbb{R}^n \to \mathbb{R}^{n_r}$ and constraints $c\colon \mathbb{R}^n \to \mathbb{R}^{n_c}$ functions are assumed to be twice continuously differentiable with respective Jacobian J and C. We may also write the least-squares objective as a general function f with gradient

$$\nabla f(x) = J(x)^\top r(x).$$

The set Ω consists of linear constraints and is defined as

$$\Omega = \{x \in \mathbb{R}^n \mid Ax = b,\ l \le x \le u\}, \tag{1.2}$$

with $A \in \mathbb{R}^{m\times n}$, $m < n$, $b \in \mathbb{R}^m$, and $l, u \in \mathbb{R}^n$. Without loss of generality, some components of the latter two vectors can be set to $\pm\infty$ for unbounded parameters. For convenience, we also

*University of Montreal, Department of Computer Science and Operations Research, Montreal, QC, Canada
†Hydro-Québec, Montreal, QC, Canada
‡Université du Québec à Montréal, Montreal, QC, Canada

assume that the linear equalities do not involve single-variable constraints. Formulation (1.1) is also suitable for problems with nonlinear inequality constraints of the form $g(x) \geq 0$. The latter are transformed into equality constraints by adding non-negative slack variables, which gives the new constraints

$$g(x) - \nu = 0, \quad \nu \geq 0.$$

The lower and upper bounds associated to these slack variables are thus 0 and ∞ respectively. Linear inequality constraints can be converted into equalities similarly.

The Lagrangian for problem (1.1), with respect to the nonlinear constraints, is defined by

$$\mathcal{L}(x, \lambda) = f(x) + \langle \lambda, c(x) \rangle ,$$

where $\lambda \in \mathbb{R}^{n_c}$ is the vector of Lagrange multipliers and $\langle \cdot, \cdot \rangle$ is the standard inner product.

An algorithm for solving problem (1.1) aims to find a first-order critical point (x_*, λ_*) that satisfies the necessary condition

$$\begin{aligned} & x_* \in \Omega \text{ and } c(x_*) = 0 \\ & P_\Omega \left[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*) \right] = x_*, \end{aligned}$$

where $P_\Omega [\cdot]$ is the projector operator onto Ω, defined, for any vector x, as

$$P_\Omega [x] = \operatorname*{argmin}_{v \in \Omega} \ \|v - x\|^2.$$

The NLS structure arises naturally in data fitting, parameter estimation, and scientific computing, and its efficient resolution has attracted a sustained research effort [13, 25, 48]. A distinctive feature of NLS problems is that the Hessian of the objective decomposes into a first-order part $J(x)^\top J(x)$, where $J(x)$ is the Jacobian of r, and second-order part involving the individual Hessians of the residuals weighted by their values. Exploiting this decomposition is the central challenge in designing efficient algorithms. The Gauss–Newton (GN) method [25, Chapter 9] retains only the first-order term, which is effective when residuals are small at the solution but can lead to poor convergence otherwise. The Levenberg–Marquardt (LM) method, initiated in [38, 44] and further studied, for instance, in [7, 9], regularizes the GN matrix by a scalar multiple of the identity, providing robustness at the cost of potentially slower convergence on large-residual problems. A richer class of approaches, known as structured quasi-Newton (SQN) methods, approximates the second-order part through secant equations tailored to the least-squares structure. The NL2SOL package [26, 37] is a popular implementation of this idea, complementing the GN term with a structured quasi-Newton correction updated via a dedicated secant equation. This algorithm also switches between the GN model or the full Hessian depending on the quality of the step. Other hybrid strategies with switching criterion based on the residual magnitude have also been explored in [1, 28, 65]. Further SQN formulations, including factorized updates and sizing strategies, have been proposed in [4, 10, 36, 63, 64]. A comprehensive numerical comparison of these techniques can be found in [42].

Extending structured NLS methods to the constrained setting has received comparatively less attention, though it remains an active area of research. Such formulations arise, for example, in data-fitting problems where the constraints enforce the physical consistency of an underlying model [24, 33, 53]. The convex case, especially the bound constrained case, is studied in [5, 6, 31, 51]. For more general nonlinear constraints, most methods operate within a sequential quadratic programming (SQP) framework. In that paradigm, each iteration solves a quadratic subproblem obtained by linearizing the constraints and approximating the Hessian

of the Lagrangian. [39] propose an SQP method whose Hessian approximation is based on a secant equation derived from [4]; [60] use a BFGS update of the projected Hessian of a merit function; and [43] update the structured approximation projected Hessian in the null space of the active constraints. For problems with equality constraints only, more recent work includes regularized approaches: [9] combine a Levenberg–Marquardt approximation of the Lagrangian Hessian with a trust-region method, and [50] derive a regularized KKT system by adding primal and dual regularization terms. Structured approximations of the Hessian within general constrained formulations have also been studied from a theoretical perspective in [59].

When the constraints can be split into two categories, as in formulation (1.1), an alternative to SQP methods is the augmented Lagrangian (AL) framework, also known as the method of multipliers, independently introduced by [34] and [52], with further theoretical foundations provided by [54]. Rather than solving a sequence of quadratic subproblems, AL methods incorporate the nonlinear constraints into the objective as a penalty term and solve a sequence of subproblems in which only the easier constraints remain. For problem (1.1), this amounts to approximately minimizing, at each outer iteration, the AL function over the polyhedral set Ω, thereby reducing the problem to a linearly constrained one. This strategy is particularly attractive as the linear part can be handled directly and efficiently by gradient projection techniques, without sacrificing the ability to enforce nonlinear constraints. The theoretical underpinnings of this approach, including global convergence results, are developed in [15, 17, 20, 21] and summarized in [22, Chapter 14]. A major implementation of this philosophy is the LANCELOT solver [19], which handles bound-constrained subproblems using a gradient projection technique [16]. More recent AL implementations have broadened this framework in several directions: [2] establish convergence under weaker constraint qualifications; [29] propose a primal-dual AL method that behaves like a stabilized SQP method in the local regime; [3] implement a matrix-free AL approach and [23] develop an adaptive AL algorithm with inexact subproblem solves. For the inner minimization that arises at each outer AL iteration, gradient projection methods [40, 46] are particularly effective for bound-constrained problems as they allow for rapid changes in the active set throughout the iterations. The linear equality case can be handled efficiently, as the projections can be computed by solving a system of normal equations [32]. When both bounds and linear equalities are present, as in our setting, direct projection onto Ω is not available in closed form. Nevertheless, we will see in the next section that the norm of the reduced gradient provides a practical criticality measure, as used in solvers such as MINOS [47] and LSNNO [61].

The present work combines two well-established lines of research: the augmented Lagrangian framework of Conn, Gould, and Toint [15, 17, 20, 21, 22], with its mature convergence theory and gradient projection inner minimization, and the quasi-Newton approximations that exploit the sum-of-squares structure of both the objective and the penalty term, following the secant equation philosophy of [4, 26] adapted to the AL setting in the spirit of [39]. Bringing these together yields a solver specifically designed for NLS problems while handling constraints of general form, including nonlinear equalities and inequalities alongside linear equalities and bounds. Our algorithm is implemented in the Julia programming language [11] and is open-source.

We end this section by introducing some notations. Jacobian matrices of r and c evaluated at x are respectively noted $J(x)$ and $C(x)$. Components of a vector $x \in \mathbb{R}^n$ are noted x_i for $i = 1, \ldots, n$. Vectors e_i denote the columns of the $n \times n$ identity matrix. When describing iterative processes to solve problem (1.1), we index vectors and matrices by the iteration number. For instance, at iteration k, (x_k, λ_k) is the current primal-dual iterate. For functions evaluated at

x_k, we use the shorthand notation $r_k = r(x_k)$, $c_k = c(x_k)$, $J_k = J(x_k)$ etc. To not interfere with previous notation for components, we add parentheses to make the distinction clear. For instance, $(x_k)_i$ denotes the i-th component of vector x_k. For any sequence $(u_k)_k$, the limit of a converging subsequence indexed by $\mathcal{K} \subset \mathbb{N}$ will be written $\lim_{k \in \mathcal{K}} u_k$. The orthogonal complement of a subspace $V \subset \mathbb{R}^n$, i.e. the set $\{w \mid \langle w, v \rangle = 0 \text{ for all } v \in V\}$, will be written $V^\perp$.

The rest of the paper is organized as follows. In Section 2, we present our algorithmic framework and discuss implementation details. We study global convergence in Section 3. Numerical experiments are shown in Section 4 and we conclude with perspectives.

2 Algorithm

2.1 Augmented Lagrangian framework

We formally state the first assumptions about problem (1.1).

Assumption 1. *Residuals r and constraints c are twice continuously differentiable on $\mathbb{R}^n$.*

Assumption 2. *The equality constraint matrix A is full row rank.*

Assumption 3. *The feasible region for the linear constraints Ω is non-empty.*

Our method is based on the Augmented Lagrangian (AL) function defined as

$$\Phi(x, \lambda, \mu) := \frac{1}{2}\|r(x)\|^2 + \langle \lambda, c(x) \rangle + \frac{\mu}{2}\|c(x)\|^2, \tag{2.1}$$

where $\mu > 0$ is a penalty parameter.

We maintain the linear constraints in the formulation of the problem and only penalize the violation of the nonlinear constraints. One has the following expression for the gradient:

$$\nabla_x \Phi(x, \lambda, \mu) = J(x)^\top r(x) + C(x)^\top \bar{\lambda}(x, \lambda, \mu),$$

where

$$\bar{\lambda}(x, \lambda, \mu) := \lambda + \mu c(x), \tag{2.2}$$

are the first-order estimates of the Lagrange multipliers. The AL and Lagrangian gradients are related by the identity

$$\nabla_x \Phi(x, \lambda, \mu) = \nabla_x \mathcal{L}(x, \bar{\lambda}(x, \lambda, \mu)). \tag{2.3}$$

The Hessian is given by

$$\nabla^2_{xx} \Phi(x, \lambda, \mu) = J(x)^\top J(x) + \mu C(x)^\top C(x) + S(x)$$

with the second-order terms explicitly given by

$$S(x) = \sum_{i=1}^{n_r} r_i(x) \nabla^2 r_i(x) + \sum_{i=1}^{n_c} \nabla^2 c_i(x) \bar{\lambda}_i(x, \lambda, \mu). \tag{2.4}$$

For fixed λ and μ, reformulating problem (1.1) with function (2.1) gives the linearly constrained problem

$$\begin{aligned} \min_x \quad & \Phi(x, \lambda, \mu) \\ \text{s.t.} \quad & x \in \Omega. \end{aligned} \tag{2.5}$$

Moving the nonlinear constraints into the objective simplifies the problem because only linear constraints are left, which enables one to use iterative methods for linearly constrained optimization while still improving feasibility of the nonlinear constraints. Since Ω is a convex set, a first-order critical point of (2.5) can also be characterized by the condition

$$P_\Omega \left[x^* - \nabla_x \Phi(x^*, \lambda, \mu) \right] = x^*.$$

In our methods, and as in standard AL methods, each iterate x_k is an approximate solution of a subproblem of the form (2.5) satisfying

$$\| P_\Omega \left[x_k - \nabla_x \Phi(x_k, \lambda_k, \mu_k) \right] - x_k \| \leq \omega_k, \tag{2.6}$$

for a fixed vector of multipliers λ_k, penalty parameter μ_k and tolerance ω_k. If the iterate x_k improves the feasibility violation of the nonlinear constraints, a dual ascent step is taken on the Lagrange multipliers by setting

$$\lambda_{k+1} = \lambda_k + \mu_k c_k = \bar{\lambda}(x_k, \lambda_k, \mu_k).$$

If on the contrary, feasibility has not been improved, the minimization (2.5) is restarted using an increased penalty parameter $\mu_{k+1} > \mu_k$ to enforce feasibility. The method is outlined in Algorithm 1. Following our notation for iteration dependent quantities, we respectively write $\nabla_x \Phi(x_k, \lambda_k, \mu_k)$ and $\bar{\lambda}(x_k, \lambda_k, \mu_k)$ in the compact form $\nabla_x \Phi_k$ and $\bar{\lambda}_k$. One first notices that

Algorithm 1 Augmented Lagrangian algorithm

Require: Initial starting point $x_0^s \in \Omega$, Lagrange multipliers estimate λ_0, penalty parameter μ_0

1: Penalty parameter increase factor $\tau > 1$, tolerance update constants $\omega, \eta, \kappa_\omega, \kappa_\eta, \beta_\omega, \beta_\eta$
2: Set initial tolerances $\omega_0 \leftarrow \omega \mu_0^{-\kappa_\omega}$ and $\eta_0 \leftarrow \eta \mu_0^{-\kappa_\eta}$
3: **for** k=0,1,2... **do**
4: Starting from x_k^s, approximately solve $\min_{x \in \Omega} \Phi(x, \lambda_k, \mu_k)$ to find $x_k \in \Omega$ satisfying (2.6).
5: **if** $\|c(x_k)\| \leq \eta_k$ **then**
6: **if** $\| P_\Omega \left[x_k - \nabla_x \Phi_k \right] - x_k \| \leq \omega_*$ and $\|c(x_k)\| \leq \eta_*$ **then**
7: **stop** and **return** approximate solution (x_k, λ_k)
8: **end if**
9: Update the Lagrange multipliers $\lambda_{k+1} \leftarrow \bar{\lambda}_k$
10: Set the next starting point $x_{k+1}^s \leftarrow x_k$
11: Leave the penalty parameter unchanged $\mu_{k+1} \leftarrow \mu_k$
12: Decrease the tolerances $\omega_{k+1} \leftarrow \omega_k \mu_{k+1}^{-\beta_\omega}$ and $\eta_{k+1} \leftarrow \eta_k \mu_{k+1}^{-\beta_\eta}$
13: **else**
14: Let the iterate unchanged $\left(x_{k+1}^s, \lambda_{k+1}\right) \leftarrow (x_k^s, \lambda_k)$
15: Increase the penalty parameter $\mu_{k+1} \leftarrow \tau \mu_k$
16: Update tolerances $\omega_{k+1} \leftarrow \omega \mu_{k+1}^{-\kappa_\omega}$ and $\eta_{k+1} \leftarrow \eta \mu_{k+1}^{-\kappa_\eta}$
17: **end if**
18: **end for**

Algorithm 1 has an outer-inner iteration structure, in the sense that each iteration requires to approximately solve an optimization problem (line 4), which also involves an iterative process. The techniques used for the latter are detailed in Section 2.2.

Tolerances in Algorithm 1 are updated in such a way that both sequences ω_k and η_k tend to 0 when $k \to \infty$. The update scheme that we outline follows the rules described in [22, Chapter 14]. The parameters values used in our implementation are given in Section 2.2.3.

We describe the convergence in terms of the norm of the projected gradient but this quantity is not easily computable in practice. Indeed, contrary to the case where there is only bounds or linear equality constraints, there is no direct formula for the projection on a polyhedral set of the form Ω. For our implementation, we will use the norm of the reduced gradient, which we define now.

For $x \in \Omega$, $I^+(x)$, resp. $I^-(x)$ denotes the indices of upper bounds, resp. lower bounds, satisfied as equalities (active) at x and $I(x) := I^+(x) \cup I^-(x)$, i.e.:

$$i \in I(x) \implies x_i \in \{l_i, u_i\}.$$

We also introduce the notion of tangent space at a point x, written $T(x)$ and defined as the set of directions d such that the same constraints are satisfied with equality at both x and $x+d$. In our case, this corresponds to the linear equality constraints and the active bounds. Formally:

$$T(x) := \{d \in \mathbb{R}^n \mid Ad = 0,\ d_i = 0 \text{ for all } i \in I(x)\}. \tag{2.7}$$

At a given point (x, λ) and parameter μ, the reduced gradient is defined as

$$P_{T(x)}\left[\nabla_x \Phi(x, \lambda, \mu)\right].$$

Provided that the multipliers associated to the active bounds are of appropriate sign, the quantity $\|P_{T(x)}\left[\nabla_x \Phi(x, \lambda, \mu)\right]\|$ is an appropriate criticality measure used in optimization algorithms for linearly constrained optimization such as MINOS [47] and LSNNO [61]. Projections on $T(x)$ can be computed in closed form. Indeed, assuming that $I(x) = \{i_1, \ldots, i_p\}$ with $p < n - m$ and denoting by $Z \in \mathbb{R}^{p \times n}$ the matrix whose row k is the row i_k of the $n \times n$ identity matrix, $T(x)$ is nothing but the null space of the block matrix

$$\tilde{A} = \begin{pmatrix} A \\ Z \end{pmatrix}. \tag{2.8}$$

Let $\tilde{N}$ be a matrix whose columns form an orthonormal basis of the null space of $\tilde{A}$. By construction and our full rank Assumption 2, $\tilde{A}$ is also full rank. Then, the projection of a vector v onto the null space of $\tilde{A}$ is given by $\tilde{P}v$ where

$$\tilde{P} = \tilde{N}\tilde{N}^\top,$$

There is no need to explicitly form the matrix $\tilde{N}$ because we can make use of the equivalent expression of the projection matrix $\tilde{P}$:

$$\tilde{P} = I - \tilde{A}^\top \left(\tilde{A}\tilde{A}^\top\right)^{-1} \tilde{A}. \tag{2.9}$$

Using (2.9), one can compute the projection of a vector v, i.e. $\tilde{P}v$, by first solving, for an auxiliary variable y, the linear system

$$\left(\tilde{A}\tilde{A}^\top\right) y = \tilde{A}v, \tag{2.10}$$

and retrieve the projection by

$$v - \tilde{A}^\top y.$$

This approach, referred as the *normal equations* approach [32], requires to solve the system (2.10). This is done by using the Cholesky decomposition of $\tilde{A}\tilde{A}^\top$. The latter is well defined because the matrix $\tilde{A}\tilde{A}^\top$ is symmetric by construction and positive definite by Assumption 2. System (2.10) is reduced to two successive lower triangular systems. The factorization $\tilde{A}\tilde{A}^\top = \tilde{L}\tilde{L}^\top$, with $\tilde{L}$ lower triangular, does not need to be computed from scratch because the structure of $\tilde{A}$ implies a block structured Cholesky factor that involves the Cholesky decomposition of $AA^\top$. The latter is constant throughout the algorithm, so the factor $\tilde{L}$ can be computed in a relatively efficient manner, even though it potentially has to be updated at every new minor iteration. We expect it to be the case when these computations take place in the first outer iterations of Algorithm 1, but that as we progress toward a first-order critical point, the set of active bounds tends to stabilize and remain the same during the inner iterations. Details on this computation can be found in Appendix A.

This discussion justifies why our implementation uses condition

$$\|P_{T_k}\left[\nabla_x \Phi_k\right]\| \leq \omega_k \tag{2.11}$$

in Algorithm 1 instead of (2.6).

2.2 Inner minimization process

In this section, we describe the inner minimization process used to produce the outer iterates of Algorithm 1. Since the current multipliers estimates λ and the penalty parameter μ are fixed, the objective function for the inner minimization only depends on x so we use the shorthand notation $\varphi : x \mapsto \Phi(x, \lambda, \mu)$. Given a point $x_0 \in \Omega$, multipliers estimates λ, a penalty parameter μ, we aim to find an approximate solution of the problem

$$\begin{aligned} \min_x \quad & \varphi(x) \\ \text{s.t.} \quad & x \in \Omega, \end{aligned} \tag{2.12}$$

and such that

$$\|P_{T(x)}\left[\nabla\varphi(x)\right]\| \leq \omega,$$

for a tolerance $\omega > 0$. This task also involves an iterative process, for which the iterates will be indexed with subscript k but are distinct from the outer iterates of Algorithm 1. At each iteration k, we compute a step s_k and recursively update the iterate by $x_{k+1} = x_k + s_k$. The step is obtained after minimizing a model of the objective function φ around x_k. We consider the quadratic model:

$$m_k(x) = \varphi_k + \langle g_k, x - x_k\rangle + \frac{1}{2}\left\langle x - x_k, H_k(x - x_k)\right\rangle, \tag{2.13}$$

where $\varphi_k := \varphi(x_k)$, $g_k := \nabla\varphi_k$ and H_k is a symmetric approximation of the true Hessian $\nabla^2_{xx}\varphi_k$. Using the latter would impact negatively the performance of the algorithm because computing the second order terms (2.4) requires too much time and storage in practice. We discuss the aspects of the methods relative to the choice of approximation in Section 2.2.2. Model (2.13) is appropriate to describe the algorithm in terms of the iterate. When discussing

subproblems, we prefer to emphasize the step and would then use the following model of the primal reduction $\varphi(x_k + s) - \varphi_k$, given by

$$q_k(s) = \frac{1}{2} \langle s, H_k s \rangle + \langle g_k, s \rangle .$$

One has $q_k(x - x_k) = m_k(x) - m_k(x_k)$.

We seek to compute the step s_k as an approximate solution of the program

$$\begin{aligned} \min_s \quad & q_k(s) \\ \text{s.t.} \quad & x_k + s \in \Omega \\ & \|s\|_\infty \leq \Delta_k, \end{aligned}$$

Vector s denotes the unknown of the subproblem whose solution s_k is the step and is used to compute the new iterate $x_{k+1} = x_k + s_k$.

Since x_0 is feasible, the constraints $x_k + s \in \Omega$ are satisfied at every iteration as long as the steps satisfy

$$As = 0, \qquad x_k - l \leq s \leq u - x_k,$$

for all k.

In our method, we also incorporate a trust-region strategy to control and assert the quality of a step. Therefore, we add to each subproblem the constraint $\|s\|_\infty \leq \Delta_k$ with a radius $\Delta_k > 0$. This constraint reflects the domain on which we believe that the model well approximates the true function. Since the ℓ_∞ trust region constraint is equivalent to imposing $x_i \in [-\Delta_k, \Delta_k]$ for all i, we can rewrite this subproblem

$$\begin{aligned} \min_s \quad & q_k(s) && \text{(2.14a)} \\ \text{s.t.} \quad & As = 0 && \text{(2.14b)} \\ & l_k \leq s \leq u_k, && \text{(2.14c)} \end{aligned}$$

where l_k and u_k are iteration-dependent bounds having components $(l_k)_i = \max(-\Delta_k, l_i - (x_k)_i)$ and $(u_k)_i = \min(\Delta_k, u_i - (x_k)_i)$ for all $i = 1, \ldots, n$.

The success of an iteration and the update of the trust region are evaluated by the ratio

$$\rho_k = \frac{\varphi(x_k + s_k) - \varphi_k}{q_k(s_k)}. \tag{2.15}$$

We follow the standard step acceptance criteria [22], which require constants $\eta_1, \eta_2, \gamma_1, \gamma_2$ such that

$$0 < \eta_1 \leq \eta_2 < 1 \text{ and } 0 < \gamma_1 \leq \gamma_2 < 1. \tag{2.16}$$

If $\rho_k > \eta_1$, the step is accepted and the trust region is expanded. Otherwise, the step is rejected and the region reduced. A typical scheme to update the radius Δ_k would be to set

$$\Delta_{k+1} \in \begin{cases} [\Delta_k, \infty) & \text{if } \rho_k \geq \eta_2 \\ [\gamma_2 \Delta_k, \Delta_k] & \text{if } \rho_k \in [\eta_1, \eta_2) \\ [\gamma_1 \Delta_k, \gamma_2 \Delta_k] & \text{if } \rho_k < \eta_1 \end{cases} \tag{2.17}$$

The procedure employed to solve subproblem (2.14) is described in Algorithm 2 and the latter is analyzed in the rest of this section.

Algorithm 2 Trust region inner iteration algorithm

Require: initial point $x_0 \in \Omega$, radius $\Delta_0 > 0$, constants $\eta_1, \eta_2, \gamma_1, \gamma_2$ satisfying conditions (2.16).

set $k \leftarrow 0$
repeat
  compute the model m_k
  compute a step s_k that sufficiently reduces the model
  compute the ratio ρ_k (2.15)
  if $\rho_k > \eta_1$ **then** set $x_{k+1} \leftarrow x_k + s_k$
  else set $x_{k+1} \leftarrow x_k$
  end if
  set Δ_{k+1} according to (2.17)
  increment $k \leftarrow k + 1$
until $\|P_{T_k}[g_k]\| \leq \omega$

2.2.1 Computation of the step

We consider the current feasible iterate x_k and its associated approximate quadratic model m_k (2.13). The step is computed in two phases. We first compute a Cauchy step s_k^C that ensures a decrease of the objective function sufficient to establish global convergence of the inner minimization algorithm. Next, we further minimize the objective function by exploring the subspace defined by the constraints active at the Cauchy point $x_k^C := x_k + s_k^C$. This approach is of common use in gradient projection techniques [48, Chapter 16] and there are different strategies to compute a Cauchy point [16, 40, 46]. Because of the structure of our constraints, we cannot directly project on the whole set Ω in practice but we can exploit prior knowledge on the active bounds. That is the reason why we compute our Cauchy step by finding the first local minimizer of the model along the projected gradient path

$$s_k(t) = P_\Omega[x_k - tg_k] - x_k \text{ for } t \geq 0. \tag{2.18}$$

Our procedure is an adaptation to the polyhedral case of algorithm SBMIN [16] used for the inner minimization phase of LANCELOT solver [19].

Assume we have found a Cauchy step s_k^C. In order to have an efficient algorithm, we want the total step to achieve a better reduction than the Cauchy step. To do so, we build the next iterate x_{k+1} after a finite sequence of M minor iterates $x_{k,1}, \ldots, x_{k,M+1}$. The sequence starts at the Cauchy point and ends at the next iterate, i.e. $x_k + s_k^C = x_{k,1}$ and $x_{k+1} = x_{k,M+1}$. This type of approach has shown to be effective for general optimization with bound constraints [40] and has also been incorporated into other AL algorithms for the inner loop minimization [3, 19].

Each minor iterate is defined after the previous one and is decomposed into

$$x_{k,j+1} = x_{k,j} + w_{k,j},$$

where $w_{k,j}$ is a descent direction for the quadratic model q_k. For each minor iterate, we require

$$x_{k,j} \in \Omega, \quad \|x_{k,j} - x_k\|_\infty \leq \Delta_k, \quad I(x_k^C) \subseteq I(x_{k,j}). \tag{2.19}$$

The first two conditions are merely that each minor iterate is feasible and the associated step lies within the trust region, while the last condition means that we can only add active bounds

during this process. Note that in this context, a bound can become active with respect to the trust region and not only the original bounds on the variables. We also require the sufficient decrease between two successive minor iterates

$$m_k(x_{k,j+1}) \leq m_k(x_{k,j}), \qquad j = 1, \ldots, M. \tag{2.20}$$

After each minor iteration, the corresponding step is $s_{k,j} := x_{k,j} - x_k$.

The search direction $w_{k,j}$ is an approximate minimizer of the subproblem

$$\min_w \quad m_k(x_{k,j} + w) \tag{2.21a}$$

$$\text{s.t.} \quad Aw = 0 \tag{2.21b}$$

$$w_i = 0, \qquad i \in I(x_{k,j}). \tag{2.21c}$$

At a minor iteration, the free variables, indexed by $\mathcal{F}(x_{k,j})$, are implicitly subject to the bounds

$$\left(l_{(k,j)}\right)_i \leq w_i \leq \left(u_{(k,j)}\right)_i, \qquad i \in \mathcal{F}(x_{k,j}) \tag{2.22}$$

where $l^{(k,j)} := l_k - s_{k,j}$ and $u^{(k,j)} := u_k - s_{k,j}$.

The minor subproblem (2.21) is solved by applying the projected conjugate gradient [32] method with the previous minor iterate $x_{k,j}$ as a starting point. Three termination cases can occur. First, a direction can be generated such that one of the bounds (2.22) is violated for one of the free variables. When this happens, the search direction is scaled so that the associated component lies at the bound and the CG iterations stop. The second case occurs when we generate a direction of negative curvature and is handled similarly to the first one, i.e. we modify the direction so that all the variables remain within their bounds. The third case is the normal termination one and occurs when a local minimizer – with respect to a given tolerance – has been found. In all cases, we return the obtained search direction $w_{k,j}$, perform the projected line search to compute the point $x_{k,j+1}$ such that eqs. (2.19) and (2.20) are verified. The handling of the termination cases differs in the continuation of the procedure. The last two cases (negative curvature and local minimality), cause the stopping of the minor iterates loop. On the contrary, if the CG iterations stopped because a bound was hit, we go back to solving (2.21) using projected conjugate gradient method with $x_{k,j+1}$ as a starting point and $I(x_{k,j+1}) \supset I(x_{k,j})$ as a new set of fixed components.

Each iteration of the conjugate gradient method applied to (2.21) requires computing projections onto the null space of the constraints (2.21b)–(2.21c). We denote the associated projection operator, in this case a $n \times n$ matrix, by $\tilde{P}$. The specification of the projected conjugate gradient method applied to solving (2.21) is outlined in Algorithm 3. The projections occurring at lines 1 and 17, and the associated operators, are computed with the normal equations approach described in previous section.

An interesting feature would be to complement each search direction by a steplength computed by a projected search [40, 46]. This allows to add more than one constraint at each minor iteration to the active set. The downside is that it would also require to compute the projection of the gradient direction (2.18) on the feasible set Ω at every new trial value of the steplength. When Ω only contains bound constraints, the projection is trivial and cheap to compute. The linear equality case is more costly but can still be handled efficiently by a *normal equations* or *augmented system* approach [32]. When Ω is of the form (1.2), the projection is less trivial and requires to solve the associated minimum-distance quadratic program with a dedicated solver for quadratic programming, which we prefer to avoid.

Algorithm 3 The projected conjugate gradient method applied to (2.21)

Set $w \leftarrow 0,\ r \leftarrow H_k(x_{k,j} - x_k) + g_k,\ v \leftarrow \tilde{P}r,\ p \leftarrow -v$
Set tolerance $\varepsilon_{cg} \leftarrow \kappa_{cg}\|v\|$
repeat
    if $\langle p, H_k p\rangle \leq 0$ **then**
        Set $w^+ \leftarrow w + \gamma p$ where γ is the smallest factor such that $w_i^+ \in \{l_i^{(k,j)}, u_i^{(k,j)}\}$
        for $i \in \mathcal{F}(x_{k,j})$
        STOP
    end if
    Set $\alpha \leftarrow \langle r, v\rangle \,/\, \langle p, H_k p\rangle$
    Set $w^+ \leftarrow w + \alpha p$
    if w^+ violates a bound **then**
        $w^+ \leftarrow w + \gamma p$ where γ is the smallest factor such that $w_i^+ \in \{l_i^{(k,j)}, u_i^{(k,j)}\}$
        for $i \in \mathcal{F}(x_{k,j})$
        STOP
    end if
    Set $r^+ \leftarrow r + \alpha H_k p$
    Set $v^+ \leftarrow \tilde{P}r^+$
    if $\sqrt{\langle r^+, v^+\rangle} < \varepsilon_{cg}$ **then STOP**
    end if
    Set $\beta \leftarrow \langle r^+, v^+\rangle \,/\, \langle r, v\rangle$
    Set $p \leftarrow -v^+ + \beta p$
    Set $w \leftarrow w^+,\ r \leftarrow r^+,\ v \leftarrow v^+$
until $2(n - m - |I(x_{k,j})|)$ iterations have been done
return w^+

We stop the minor-iterations procedure when the reduced gradient associated to the current active set is small enough. More formally, assume we performed the $j+1$ minor minimizations and let $T_{k,j}$ be the tangent space at $x_{k,j}$ with respect to the active bounds in $I(x_{k,j})$. We stop the minor iteration loop whenever the following inequality is satisfied:

$$\left\| P_{T_{k,j}} \left[\nabla m_k(x_{k,j+1}) \right] \right\| \leq \kappa_{mlt} \left\| P_{T_{k,j}} \left[\nabla m_k(x_k) \right] \right\|,$$

with $\kappa_{mlt} \in (0,1)$. This inequality estimates if there is relative progress that can be made in the tangent subspace spanned by the free variables.

2.2.2 Hessian approximation

We now discuss how the Hessian of the AL is iteratively updated when we define the quadratic model of iteration k in Algorithm 2. To set up the context and notations of this paragraph, we wish to approximate the true Hessian

$$\nabla^2_{xx}\varphi(x_k) = J_k^\top J_k + \mu C_k^\top C_k + S_k,$$

by a symmetric matrix H_k. We remind that the need for an approximation mainly comes from the fact that evaluating the second order terms in S_k requires too much time and storage to be done in practice, especially for problems with a large number of variables and residuals.

The first approximation we can think of, and the simplest one, is obtained after merely linearizing the residuals and constraints in expression (2.1), which gives

$$H_k = J_k^\top J_k + \mu C_k^\top C_k. \tag{2.23}$$

We also call it the Gauss-Newton (GN) approximation, in reference to its counterpart in the unconstrained case. On the one hand, this approximation is very convenient and cheap as it involves already available first-order derivatives, since they are required to evaluate the gradient. Also, when the Jacobians are full rank, the resulting matrix is positive-definite which guarantees the convexity of subproblems of the form (2.12). On the other hand, it is known to have a slower rate of convergence on problems with non zero residuals at the solution. This downside is amplified when we are to approximate the Hessian of the Lagrangian, or the AL in our case. Indeed, setting S_k to the zero matrix not only neglects the contribution of the residuals to the curvature of the model, but also neglects the contribution of the Lagrange multipliers, for which there are no reasons to equal zero.

We thus need to take into account the full Hessian to form an approximation. Looking at the literature on this subject, one can observe that there is a variety of approaches focusing on the unconstrained case, as it is a major challenge in improving the efficiency of algorithms for problems with non zero residuals at the solution. Nevertheless, relevant parallels can be drawn with the AL situation, or the constrained case in general, since these studies explore techniques to deal with second order terms. Because the first-order terms are readily available and provide curvature information, only the second-order components need to be approximated. Therefore, we look for an approximation of the form

$$H_k = B_k^{GN} + B_k. \tag{2.24}$$

where B_k^{GN} is the right hand side of (2.23) and B_k is a symmetric approximation of S_k.

Structured approximations of the AL Hessian have been studied for general objective functions [59] but our approach is closer to the one employed in [39]. In the latter, the authors base

their approximation on a secant equation derived from a heuristic initially introduced in the unconstrained case [4] that they adapted to approximate the Hessian of the Lagrangian and use it in a SQP algorithm for constrained nonlinear least-squares. This strategy is also at the heart of the SQN method from the popular package for unconstrained nonlinear least-squares NL2SOL [26, 37]. The idea is the following. If we were to approximate each matrix term of the sum (2.4), we would have

$$B_{k+1} = \sum_{i=1}^{n_r} r_i(x_{k+1}) B_{k+1}^{r_i} + \sum_{i=1}^{n_c} \bar{\lambda}_i(x_{k+1}, \lambda, \mu) B_{k+1}^{c_i}, \tag{2.25}$$

with $B_{k+1}^{r_i} \approx \nabla^2 r_i(x_{k+1})$ and $B_{k+1}^{c_i} \approx \nabla^2 c_i(x_{k+1})$. To get an accurate approximation, given a step s_k, it is reasonable to require

$$B_{k+1}^{r_i} s_k = \nabla r_i(x_{k+1}) - \nabla r_i(x_k) \qquad \text{and} \qquad B_{k+1}^{c_i} s_k = \nabla c_i(x_{k+1}) - \nabla c_i(x_k), \tag{2.26}$$

i.e. that each Hessian maps the change in the variables to the change in the gradients. Noticing that, for each i, $\nabla r_i(x_{k+1}) - \nabla r_i(x_k)$, resp. $\nabla c_i(x_{k+1}) - \nabla c_i(x_k)$, is the i-th row of $(J_{k+1} - J_k)^\top$, resp. $(C_{k+1} - C_k)^\top$, summing over the residuals and constraints indices all the terms of (2.26) gives, by (2.25), the structured secant equation

$$B_{k+1} s_k = (J_{k+1} - J_k)^\top r_{k+1} + (C_{k+1} - C_k)^\top \bar{\lambda}_{k+1}. \tag{2.27}$$

To follow the conventional notations of quasi-Newton methods, we will denote the right hand side of (2.27) by $\tilde{y}_k$ to insist on its distinction with the standard term $y_k := g_{k+1} - g_k$. Note that, as in SQN methods for unconstrained least-squares [42], one has

$$B_k^{GN} s_k + \tilde{y}_k = y_k,$$

so the resulting approximation satisfies the standard secant equation $H_k s_k = y_k$.

We can now derive an update formula from equation (2.27). In our implementation, we apply the SR1 update

$$B_{k+1} = B_k + \frac{(\tilde{y}_k - B_k s_k)(\tilde{y}_k - B_k s_k)^\top}{\langle \tilde{y}_k - B_k s_k, s_k \rangle}, \tag{2.28}$$

if $\langle \tilde{y}_k - B_k s_k, s_k \rangle \neq 0$. When the denominator in (2.28) is zero, we simply set $B_{k+1} = B_k$. To avoid numerical errors, we apply the update when

$$|\langle \tilde{y}_k - B_k s_k, s_k \rangle| \geq \kappa_{sds} \|s_k\| \|\tilde{y}_k - B_k s_k\|, \tag{2.29}$$

for $\kappa_{sds} \in (0, 1)$. Inequality (2.29) is one of the most common safeguards used in SR1 methods.

The choice of the SR1 method is motivated by its robustness in the least-squares context, as the exhaustive benchmarks in [42] show, and its tendency to better approximate the true Hessian with each iteration [18]. The approximation could thus be indefinite, which could be considered as a weakness compared to BFGS or DFP updates, that are guaranteed to be positive definite as long as it is the case for the initial approximation B_0. However, as it is highlighted in Algorithm 3, the use of the conjugate gradients method in a trust region framework handles, and even exploits, the potential non-convexity of the subproblems. Moreover, this update does not require a curvature condition $\langle s_k, y_k \rangle > 0$ to be satisfied. In other works on the constrained case, SQN methods are used to approximate the projected Hessian of the Lagrangian [60], or

of a merit function [43], in the null space of the active constraints. The BFGS update is then preferred because this matrix is, under standard assumptions, positive semidefinite, according to the second-order necessary conditions.

We end our description of the Hessian approximation by discussing hybrid updates, a feature commonly used in SQN methods for nonlinear least-squares algorithms, either in the unconstrained [1, 26, 28] or constrained [39, 60] case. The idea is to test at every iteration if the problem has small or large residuals at the solution. In the latter case, a structured update is used to increase the accuracy of the approximated Hessian whereas in the former, the GN approximation is employed, as it is more likely to correspond to the true Hessian. Note that the update, such as (2.28), is still computed at every iteration to continue accumulating curvature information. We use a strategy inspired from [28] adapted to the constrained case in [60], that computes the relative reduction

$$\zeta_k = \frac{\varphi(x_k) - \varphi(x_{k+1})}{\varphi(x_k)},$$

and updates the approximated Hessian based on the rule

$$H_{k+1} = \begin{cases} B_{k+1}^{GN} & \text{if } \zeta_k \leq \kappa_{hyb} \\ B_{k+1}^{GN} + B_{k+1} & \text{otherwise,} \end{cases} \tag{2.30}$$

where κ_{hyb} is a parameter in $(0, 1)$. This update is used with $\kappa_{hyb} = 0.1$ in [60], where the AL plays the role of a merit function. As we will see in Section 4, it has an important impact on the performance of the algorithm.

2.2.3 Summary and implementation details

We end our discussion by summarizing the relations between outer, inner and minor iterates and the default values for the different constants exposed in this section.

- The outer iterates x_K are the main iterates of the algorithm and are approximate minimizers of the AL
- Each outer iterate is formed after a sequence of inner iterates $(x_k)_k$, linked by $x_{k+1} = x_k + s_k$ where s_k is an approximate solution of the QP (2.14)
- Each inner iterate is formed after a sequence of M minor iterates $x_{k,j}$ linked by $x_{k,j+1} = x_{k,j} + w_{k,j}$ where $w_{k,j}$ is a descent direction obtained by applying the projected conjugate gradient Algorithm 3

The relations between the three levels of iterations are illustrated in Figure 1. Our intention with the work presented in this paper was to conceive an algorithm able to handle directly linear constraints of the form (1.2) but we also accept problems where linear constraints are only bounds and then

$$\Omega = \{x \in \mathbb{R}^n \mid l \leq x \leq u\}.$$

The framework we have presented remains valid for this formulation. The only practical difference is in the computation of projections. In the bound constrained case, the projection of a vector x on the set Ω has components

$$\left(P_\Omega[x]\right)_i = \begin{cases} l_i & \text{if } x_i < l_i \\ x_i & \text{if } x_i \in [l_i, u_i] \\ u_i & \text{if } x_i > u_i \end{cases} \tag{2.31}$$

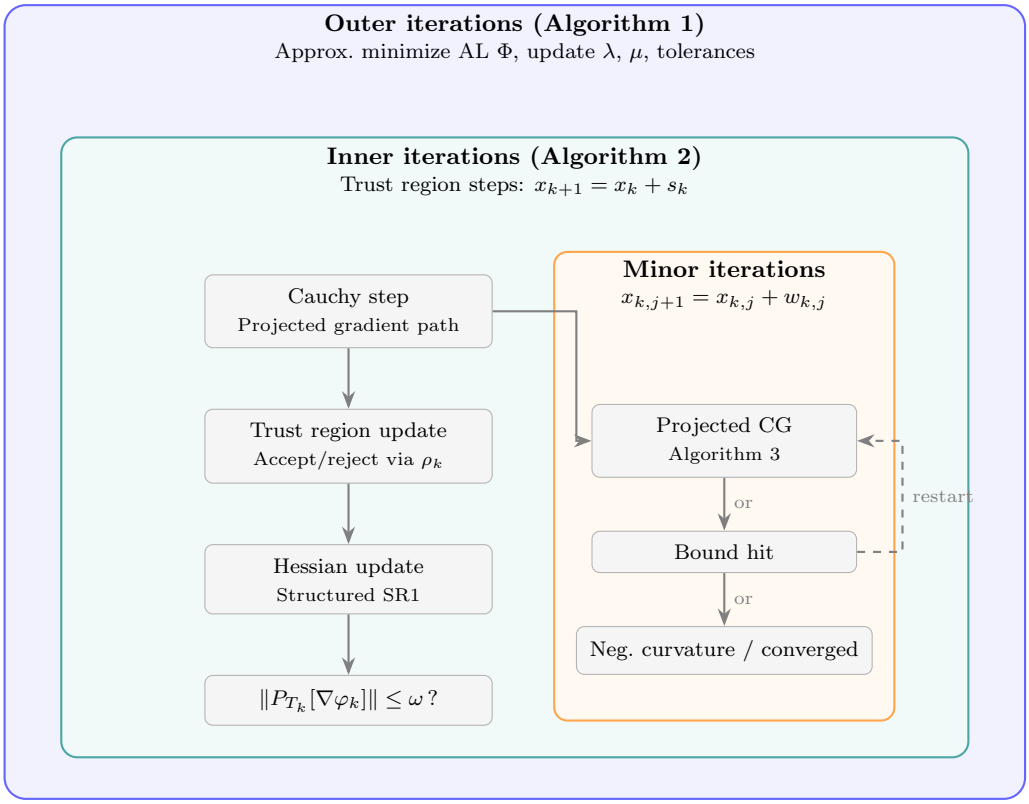

Figure 1: Hierarchy of the three iteration levels in the AL algorithm. The outer loop updates the Lagrange multipliers and penalty parameter. Each outer iteration triggers a sequence of trust-region inner iterations, and each inner iteration computes its step via a Cauchy point followed by minor iterations using the projected conjugate gradient method.

In our implementation, the only practical difference with the polyhedral case is that we can use directly the norm of the projected gradient

$$\|x_K - P_\Omega\left[x_K - \nabla_x \Phi_K\right]\|,$$

as a criticality measure.

The default values for tolerances and constants of Algorithm 1 are

$$\mu_0 = 10,\ \kappa_\omega = \beta_\omega = \eta = \omega = 1,\ \kappa_\eta = 0.1,\ \beta_\eta = 0.9,\ \tau = 100,\ \omega_* = \eta_* = 10^{-7}.$$

The constants relative to the inner minimization process of Algorithm 2 are set to the subproblem criticality tolerance, i.e. at outer iteration K, we set

$$\kappa_{cg} = \kappa_{mlt} = \omega_K.$$

The value of constant κ_{sds} used safeguard (2.29) is set to the square root of the relative double precision.

At the start of each outer iteration, we set our initial approximation of the second order terms of the Hessian to $B_0 = 0$ and the constant used to monitor the hybrid switching scheme (2.30) of the Hessian is set to $\kappa_{hyb} = 0.1$.

We now discuss the trust region handling. The initial radius value for Algorithm 2 is set to

$$\Delta_0 = 0.1\|g_0\|_\infty.$$

The mechanism to update the trust region given in (2.17) still leaves important flexibility. We choose to follow a standard strategy similar to the one exposed in [22, Chapter 17]. We also added a refinement to handle the case where the ratio ρ_k is negative, which can happen when there is very poor agreement between the function and the model. In such cases, we reduce more severely the radius by taking

$$\Delta_{k+1} = \gamma_1 \Delta_k.$$

The complete update rule for the trust region radius is then

$$\Delta_{k+1} = \begin{cases} \max\left(\alpha_2\|s_k\|_\infty, \Delta_k\right) & \text{if } \rho_k \geq \eta_2 \\ \Delta_k & \text{if } \rho_k \in [\eta_1, \eta_2) \\ \alpha_1\|s_k\|_\infty & \text{if } \rho_k \in [0, \eta_1) \\ \min\left(\alpha_1\|s_k\|_\infty, \gamma_1\Delta_k\right) & \text{if } \rho_k < 0, \end{cases}$$

with constant values

$$\alpha_1 = 0.25,\ \alpha_2 = 2.5,\ \eta_1 = 0.25,\ \eta_2 = 0.75,\ \gamma_1 = 0.0625.$$

3 Convergence analysis

In this section, we study the convergence of Algorithm 1 towards a first-order critical point of problem (1.1). We start by showing global convergence of Algorithm 2, implying that the inner minimization phase of Algorithm 1 is always well defined. We then establish global convergence of the main algorithm. Our analysis is based on the criticality measure, i.e. the norm of the projected gradient, which differs from the reduced gradient norm (2.11) used in our implementation. At the end of this section, we discuss the validity of our convergence results when (2.11) is used as a inner iteration termination condition instead of (3.9).

3.1 Global convergence of the inner minimization

Because our inner-minimization procedure closely resembles the SBMIN algorithm [16] – used in the bound-constrained inner-minimization phase of LANCELOT [19] – our proof follows the structure of the global convergence proof in [15]. Most of the work lies in reformulating and adapting the intermediate results to the polyhedral case, accounting for the structure of the linear constraints.

We first make the standard assumption

Assumption 4. *The set $\mathcal{X} = \{x \mid \varphi(x) \leq \varphi(x_0)\} \cap \Omega$ is non empty and compact.*

By Assumption 1, it is implicit that the function φ is twice continuously differentiable on $\mathcal{X}$ so we will not state this in the propositions given in this section.

We remind that the quadratic model is of the form

$$q_k(s) = \frac{1}{2}\langle s, H_k s\rangle + \langle g_k, s\rangle,$$

where $g_k = \nabla\varphi(x_k)$ and H_k is an approximation of the Hessian based on the structured SR1 formula from 2.2.2. We formulate two assumptions relative to those approximations. The norm used is the induced norm on matrices, i.e. $\|M\| := \sup_{\|x\|=1} \|Mx\|$ for a given matrix M.

Assumption 5. *Defining, for every iteration index k, the scalars b_k by*

$$b_k = 1 + \max_{0\le i\le k} \|H_i\|,$$

we require that the series $\sum_k \frac{1}{b_k}$ diverges to ∞.

The second assumption states that the norm of the approximating Hessians should not increase too fast compared with the speed of convergence of the function values.

Assumption 6.

$$\lim_{k\to\infty} b_k(\varphi_{k+1} - \varphi_k) = 0.$$

The projected gradient path is defined as

$$s_k(t) = P_\Omega\left[x_k - t g_k\right] - x_k \text{ for } t \ge 0.$$

The reduction of the model along the projected gradient path may thus be defined as the piecewise quadratic function

$$\psi(t) = q_k(s_k(t)),$$

and we denote by t_k^C the first local minimum of ψ subject to the trust region constraint

$$\|s_k(t)\|_\infty \le \Delta_k.$$

The associated Cauchy step is

$$s_k^C = s_k(t_k^C).$$

Because of the choice of the ℓ_∞ norm, computing $s_k(t)$ within the trust region corresponds to project the direction $x_k - t g_k$ onto

$$\left\{d \in \mathbb{R}^n \mid Ad = 0,\ l_k \le d \le u_k\right\}, \tag{3.1}$$

with $(l_k)_i = \max\left(-\Delta_k, l_i - (x_k)_i\right)$ and $(u_k)_i = \min\left(\Delta_k, u_i - (x_k)_i\right)$ for all $i = 1, \ldots, n$.

We assume that the total step s_k produces a fraction of the reduction achieved by the Cauchy point:

$$q_k(s_k) \le \kappa_{fcd}\ q_k(s_k^C), \tag{3.2}$$

for $\kappa_{fcd} \in (0, 1]$.

We detail the behavior of the polygonal line $s_k(t)$. Let $I(t)$ be the set of bounds l_k or u_k satisfied with equality at $s_k(t)$. Notice that this definition slightly differs from the active set $I(x)$ defined earlier, as the new formulation know takes into account the trust region. At first, if no bounds are active at x_k, projecting on the set (3.1) for t close to 0 is equivalent

to projecting onto the null space of A. As t increases, the direction might hit several bounds. Since the set of active bounds can only be increased, we have that

$$I(t) \subseteq I(t') \quad \text{for all } 0 < t \leq t'. \tag{3.3}$$

Let $0 = t_0 < t_1 < \ldots < t_p$ be the successive values of t at which the projected step $s_k(t)$ hits a bound, also called breakpoints. Hitting a bound not only affects the corresponding components, that are now fixed, but it also affects the other components, as the steepest direction must now be projected on a subspace

$$\{d \in \mathbb{R}^n \mid Ad = 0,\ d_i = 0 \text{ fixed components } i\}$$

This motivates to adapt the notion of tangent space at a point on the projected gradient path:

$$T(t) := \{d \in \mathbb{R}^n \mid Ad = 0,\ d_i = 0 \text{ for all } i \in I(t)\},$$

for $t \geq 0$. This enables us to give a recursive expression of $s_k(t)$ on each interval $[t_i, t_{i+1})$, with $0 \leq i \leq p$, as:

$$s_k(t) = (t - t_i)P_{T(t_i)}[-g_k] + s_k(t_i). \tag{3.4}$$

We also define the reduced gradient on the projected gradient path:

$$z_k(t) := P_{T(t)}[g_k]. \tag{3.5}$$

Note that the reduced gradient, and hence the path $s_k(t)$ are well defined because of the full rank Assumption 2 of the matrix A. As for the differentiability of φ, we will not remind it in the results of this section.

We now state a first lemma on how the tangent spaces and the reduced gradients at two different positions compare to each other.

Lemma 3.1. *For all $0 < t < t'$, we have that*

$$T(t) \supseteq T(t'),$$

and

$$\|z_k(t)\| \geq \|z_k(t')\|. \tag{3.6}$$

Proof. The first statement follows from the inclusion (3.3). Inequality (3.6) then results from the fact that projecting the same vector on a smaller linear subspace reduces the norm of its projection. □

When referring to the tangent space and the reduced gradient at a given breakpoint t_i, we will make use of the shorthand notation $T_i := T(t_i)$ and $z_i := z_k(t_i)$. Notice that because the active set is fixed on each interval $[t_i, t_{i+1})$, so is the tangent space and hence, the reduced gradient is constant. Also, we can deduce from Lemma 3.1 that the tangent spaces associated to each breakpoint form a finite sequence of nested linear subspaces

$$T_0 \supseteq T_1 \supseteq \ldots \supseteq T_p.$$

We can now expand equation (3.4):

$$s_k(t) = -(t - t_i)z_i - \sum_{j=0}^{i-1}(t_{j+1} - t_j)z_j, \tag{3.7}$$

which shows that on each interval (t_i, t_{i+1}), $s_k(t)$ is differentiable w.r.t. t and that

$$s_k'(t) = -z_k(t) = -z_i, \tag{3.8}$$

for $t \in (t_i, t_{i+1})$.

We start by establishing an inequality on the decrease of the objective function after taking step s_k. This will involve the norm of the projected gradient, i.e.:

$$h_k := \|s_k(1)\|. \tag{3.9}$$

Lemma 3.2. *If Assumption 4 holds and that $h_k > 0$, then*

$$\|z_k(t_k^{(1)})\| \geq \frac{h_k}{2},$$

where

$$t_k^{(1)} = \frac{h_k}{2\kappa_{ubg}},$$

and κ_{ubg} is the constant defined by $\kappa_{ubg} := \max(1, \max_{x\in\mathcal{X}} \|\nabla\varphi(x)\|)$.

Proof. First note that the constant κ_{ubg} is well defined because φ is continuously differentiable on $\mathcal{X}$, the latter being compact by Assumption 4.

Since the set Ω is closed and convex, the projection mapping $P_\Omega[\cdot]$ is non expansive, thus, for any $t \geq 0$:

$$\begin{aligned}
\|s_k(t)\| &= \|P_\Omega[x_k - tg_k] - x_k\| \\
&= \|P_\Omega[x_k - tg_k] - P_\Omega[x_k]\| \\
&\leq \|x_k - tg_k - x_k\| \\
&\leq t\|g_k\|.
\end{aligned}$$

Choosing $t = t_k^{(1)}$ and because $\|g_k\| \leq \kappa_{ubg}$ by definition of κ_{ubg}, we get

$$\|s_k(t_k^{(1)})\| \leq \frac{h_k}{2}. \tag{3.10}$$

Now, defined $t_k^{(2)}$ as the smallest $t \geq 0$ such that

$$\|s_k(t_k^{(2)})\| = h_k. \tag{3.11}$$

By (3.10), we have

$$0 < t_k^{(1)} < t_k^{(2)} \leq 1. \tag{3.12}$$

On each interval (t_i, t_{i+1}), the left, resp. right, derivative of $s_k(t)$ at t_i, resp. t_{i+1}, are well defined and equal z_i. We can thus rewrite (3.7) as

$$s_k(t) = \int_{t_i}^{t} s_k'(t)dt + \sum_{j=0}^{i-1} \int_{t_j}^{t_{j+1}} s_k'(t)dt,$$

which we simplify by

$$s_k(t) = -\int_0^t z_k(t)dt, \tag{3.13}$$

using (3.8) and keeping in mind that it is a sum of integrals defined on each segment $[t_i, t_{i+1}]$. Now let i_1, resp. i_2 be the breakpoint index such that $t_k^{(1)} \in [t_{i_1}, t_{i_1+1})$, resp. $t_k^{(2)} \in [t_{i_2}, t_{i_2+1})$. Then by (3.13):

$$s_k(t_k^{(2)}) - s_k(t_k^{(1)}) = -\int_{t_k^{(1)}}^{t_k^{(2)}} z_k(t)dt,$$

which leads to

$$\begin{aligned}
\|s_k(t_k^{(2)}) - s_k(t_k^{(1)})\| &\leq \int_{t_k^{(1)}}^{t_k^{(2)}} \|z_k(t)\|dt \\
&\leq \int_{t_k^{(1)}}^{t_k^{(2)}} \|z_k(t_k^{(1)})\|dt \\
&\leq (t_k^{(2)} - t_k^{(1)})\|z_k(t_k^{(1)})\|,
\end{aligned}$$

where we have used (3.6) to bound the norm of the reduced gradient on $[t_k^{(1)}, t_k^{(2)}]$. Combined with (3.12) and (3.11), we get

$$\begin{aligned}
\frac{h_k}{2} = \|s_k(t_k^{(2)})\| - \frac{h_k}{2} &\leq \|s_k(t_k^{(2)})\| - \|s_k(t_k^{(1)})\| \\
&\leq \|s_k(t_k^{(2)}) - s_k(t_k^{(1)})\| \\
&\leq (t_k^{(2)} - t_k^{(1)})\|z_k(t_k^{(1)})\| \\
&\leq \|z_k(t_k^{(1)})\|,
\end{aligned}$$

which is the desired inequality. □

The next lemma gives an upper bound on the quadratic model $\psi(t)$ in an interval of interest.

Lemma 3.3. *If Assumption 4 holds and that for some $t_k^{(3)} > 0$, one has*

$$\alpha_k = \|z_k(t_k^{(3)})\| > 0,$$

then, if T is the set of points in $[0, t_k^{(3)}]$ at which the piecewise quadratic ψ is differentiable,

$$\psi'(t) \leq -\alpha_k^2 + t_k^{(3)}\kappa_{ubg}^2\|H_k\|, \tag{3.14}$$

for all $t \in T$.

Furthermore,

$$\psi'(t) \leq -\frac{1}{2}\alpha_k^2,$$

for all $t \in T \cap [0, t_k^{(4)}]$ and

$$\psi(t) \leq -\frac{\alpha_k^2}{2}t,$$

for all $t \in [0, t_k^{(4)}]$ where

$$t_k^{(4)} = \min\left(t_k^{(3)}, \frac{\alpha_k^2}{2\kappa_{ubg}^2(\|H_k\| + 1)}\right). \tag{3.15}$$

Proof. For $t \in T$, let i be the breakpoint index such that $t \in (t_i, t_{i+1})$. On the latter interval, ψ is differentiable and we have, by expression (3.7):

$$\begin{aligned}\psi'(t) &= \langle g_k, s_k'(t)\rangle + \langle s_k(t), H_k s_k'(t)\rangle \\ &= -\langle g_k, z_i\rangle - \langle s_k(t), H_k z_i\rangle .\end{aligned}$$

Because z_i is, by definition, the orthogonal projection of the vector g_k on a linear subspace:

$$\begin{aligned}\langle g_k, z_i\rangle &= \langle z_i, z_i\rangle \\ &\geq \|z_k(t_k^{(3)})\|^2 = \alpha_k^2,\end{aligned} \tag{3.16}$$

where the last inequality follows from the monotonicity of the reduced gradient norm on $[0, \infty)$.

We now look at the quadratic terms. First, by Cauchy-Schwarz inequality:

$$|\langle s_k(t), H_k z_i\rangle| \leq \|s_k(t)\|\|H_k z_i\|.$$

Then, applying the triangle inequality to expression (3.7), we obtain

$$\begin{aligned}\|s_k(t)\| &\leq (t - t_i)\|z_i\| + \sum_{j=0}^{i-1}(t_{j+1} - t_j)\|z_j\| \\ &\leq (t - t_i)\|g_k\| + \sum_{j=0}^{i-1}(t_{j+1} - t_j)\|g_k\| \\ &\leq t\|g_k\| \leq t_k^{(3)}\kappa_{ubg},\end{aligned} \tag{3.17}$$

where we have used the facts that for all breakpoints indices j, $\|z_i\| \leq \|g_k\| \leq \kappa_{ubg}$ and that the scalar t is taken in T. For the remaining terms:

$$\|H_k z_i\| \leq \|H_k\|\|z_i\| \leq \kappa_{ubg}\|H_k\|.$$

Combining the latter inequality with (3.17), we get the following bound for the quadratic terms:

$$|\langle s_k(t), H_k z_i\rangle| \leq t_k^{(3)}\kappa_{ubg}^2\|H_k\|. \tag{3.18}$$

Hence, using (3.16) and (3.18):

$$\psi'(t) \leq -\alpha_k^2 + t_k^{(3)}\kappa_{ubg}^2\|H_k\|,$$

for all $t \in T$, which proves (3.14). Now, considering $t_k^{(4)}$ as defined by (3.15), since $t_k^{(4)} \leq t_k^{(3)}$, we get that $\|z_k(t_k^{(4)})\| \geq \alpha_k > 0$. The above reasoning based on $t_k^{(4)}$ thus yields

$$\psi'(t) \leq -\alpha_k^2 + t_k^{(4)}\kappa_{ubg}^2\|H_k\| \leq -\frac{\alpha_k^2}{2},$$

for $t \in T$, $t \leq t_k^{(4)}$. It follows that, for all $t \in [0, t_k^{(4)}]$:

$$\psi(t) \leq -\frac{\alpha_k^2}{2}t,$$

which completes the proof. □

We can now establish a bound on the decrease guaranteed by the step.

Theorem 3.1. *If assumptions 4, 5 hold and that $h_k > 0$, then*

$$q_k(s_k) \leq -\kappa_{mdc} h_k^2 \min\left(\frac{h_k^2}{b_k}, \Delta_k\right), \tag{3.19}$$

where

$$\kappa_{mdc} = \frac{\kappa_{fcd}}{64\kappa_{ubg}^2}.$$

Furthermore, if iteration k is successful, then

$$\varphi(x_k) - \varphi(x_k + s_k) \geq \kappa_{sdc} h_k^2 \min\left(\frac{h_k^2}{b_k}, \Delta_k\right), \tag{3.20}$$

with $\kappa_{sdc} = \eta_1 \kappa_{mdc}$.

Proof. We first observe that if we use $t_k^{(1)}$ as $t_k^{(3)}$ in the proof of Lemma 3.3 and apply Lemma 3.2, we get

$$\psi(t) \leq -\frac{h_k^2}{8} t, \tag{3.21}$$

for $t \in [0, t_k^{(5)}]$ with

$$t_k^{(5)} := \min\left(t_k^{(1)}, \frac{h_k^2}{8\kappa_{ubg}^2(\|H_k\| + 1)}\right) = \frac{h_k^2}{8\kappa_{ubg}^2(\|H_k\| + 1)}.$$

We will bound the decrease obtained with the step depending on where the point associated with $t_k^{(5)}$ lies with respect to the trust region boundary.

First, assume that $\|s_k(t_k^{(5)})\|_\infty \leq \Delta_k$. By (3.21), one has $\psi'(t) < -h_k^2/8 < 0$ for all $t \in [0, t_k^{(5)}]$, so ψ in strictly decreasing on this segment. Because the Cauchy point is the first local minimizer of ψ subject to the trust region and the trust region is not violated at $t_k^{(5)}$, we must have $t_k^C \geq t_k^{(5)}$. Therefore,

$$\psi(t_k^C) \leq \psi(t_k^{(5)}) \leq -\frac{h_k^2}{8} t_k^{(5)}$$

Then, by (3.2), $q_k(s_k) \leq \kappa_{fcd} q_k(s_k^C)$ and it follows that

$$q_k(s_k) \leq -\kappa_{fcd} \frac{h_k^2}{8} t_k^{(5)}. \tag{3.22}$$

Now, assume that $\|s_k(t_k^{(5)})\|_\infty > \Delta_k$. The latter implies that $\|s_k(t_k^C)\|_\infty = \Delta_k$ and from

$$\left\| s_k\left(\frac{\Delta_k}{\kappa_{ubg}}\right)\right\|_\infty \leq \left\| s_k\left(\frac{\Delta_k}{\kappa_{ubg}}\right)\right\| \leq \frac{\Delta_k}{\kappa_{ubg}} \|g_k\| \leq \Delta_k,$$

we can deduce that

$$t_k^C \geq \frac{\Delta_k}{\kappa_{ubg}}.$$

Therefore, using (3.21) with $t = t_k^C$ and (3.2) implies

$$q_k(s_k) \leq -\kappa_{fcd} \frac{h_k^2}{8\kappa_{ubg}} \Delta_k \leq -\kappa_{fcd} \frac{h_k^2}{64\kappa_{ubg}^2} \Delta_k, \tag{3.23}$$

because $\kappa_{ubg} \geq 1$. The inequality (3.19) results from gathering (3.22), (3.23) and the definition of scalar b_k.

Finally, if the step is successful, we get inequality (3.20) by using (3.19) and the step acceptance condition $\rho_k > \eta_1$ from Algorithm 2. □

Once we have established the guaranteed decrease inequality (3.19), the rest of the proof does not involve the polyhedral structure of the constraints and we can fall back into the standard convergence theory of trust region methods. We state the main convergence theorem, whose proof and intermediate developments can be found in [15].

Theorem 3.2 (Theorem 11 from [15])**.** *If assumptions 4, 5, 6 hold, then*

$$\lim_{k \to \infty} h_k = 0.$$

3.2 Global convergence of the algorithm

The content of this section follows the structure of the global convergence proof outlined in [21] for AL algorithms designed to solve problems whose constraints combine nonlinear equalities and linear inequalities. We first make the following assumption about the iterates considered.

Assumption 7. *The iterates produced by Algorithm 1 lie within a closed bounded domain of* $\mathbb{R}^n$.

Let $(x_k)_k$ be a sequence of such iterates, then Assumption 7 ensures the existence of a subsequence indexed by $\mathcal{K} \subseteq \mathbb{N}$ converging to a limit point x_*. Since the iterates satisfy $x_k \in \Omega$ for all k, we also have $x_* \in \Omega$. We recall from (2.7) that $I(x)$ is the set of bounds active at x and $T(x)$ the associated tangent space, and we abbreviate $I_* := I(x_*)$. We denote by $\tilde{A}_*$ the matrix gathering the linear constraints active at x_*,

$$\tilde{A}_* := \begin{pmatrix} A \\ Z_* \end{pmatrix},$$

where Z_* is the submatrix of the identity whose rows are $e_i^\top$, $i \in I_*$. Consistently with the well-posedness of the projector (2.9), $\tilde{A}_*$ has full row rank, so $T(x_*) = \text{null}(\tilde{A}_*)$ and the columns of the orthonormal matrix N_* form a basis of $T(x_*)$. The next assumption ensures that the null space of the active linear constraints is large enough to achieve feasibility of the nonlinear constraints, and that the gradients of those constraints are linearly independent within that space.

Assumption 8. *The rank of the matrix $C(x_*)N_*$ is no smaller than n_c at any limit point x_* of the sequence $(x_k)_k$.*

We now introduce the remaining notations and notions used in our convergence analysis.

Since Ω is of the form (1.2), the normal cone to Ω at $x \in \Omega$ can be defined as

$$\mathcal{N}_\Omega(x) = \left\{ A^\top \xi + \sum_{i \in I^+(x)} \sigma_i^+ e_i - \sum_{i \in I^-(x)} \sigma_i^- e_i \;\middle|\; \xi \in \mathbb{R}^m,\ \sigma_i^+, \sigma_i^- \geq 0 \right\}, \tag{3.24}$$

where the equality multipliers ξ are free in sign while the bound multipliers are signed. The first-order criticality condition $P_\Omega\left[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)\right] = x_*$ of problem (1.1) is then equivalent to

$$-\nabla_x \mathcal{L}(x_*, \lambda_*) \in \mathcal{N}_\Omega(x_*).$$

Finally, for vectors x at which the matrix $C(x)N_*N_*^\top C(x)^\top$ is non-singular, the least-squares multipliers associated with $\tilde{A}_*$ are defined by

$$\lambda(x) = -\big((C(x)N_*)^\dagger\big)^\top N_*^\top \nabla f(x),$$

where $(C(x)N_*)^\dagger = N_*^\top C(x)^\top \big(C(x)N_*N_*^\top C(x)^\top\big)^{-1}$ is the right pseudoinverse of $C(x)N_*$. When $(C(x)N_*)^\dagger$ is well defined, Assumption 1 ensures that $\lambda(x)$ is differentiable and that its Jacobian $\nabla\lambda(x)$ is bounded in a neighborhood of x_* (see [21, Lemma 2.1]).

We can now proceed with the global convergence proof, starting with a first lemma giving a bound on the reduced gradient norm.

Lemma 3.4. *Let $(x_k)_{k \in \mathcal{K}} \in \Omega$ be a sequence converging to x_* and such that*

$$\|P_\Omega\left[x_k - \nabla_x \Phi_k\right] - x_k\| \leq \omega_k,$$

with $\lim_{k \to \infty} \omega_k = 0$. Then

$$\left\|P_{T(x_*)}\left[\nabla_x \Phi_k\right]\right\| \leq \omega_k,$$

for all $k \in \mathcal{K}$ large enough.

Proof. To lighten the notations, we write $v_k = P_\Omega\left[x_k - \nabla_x \Phi_k\right]$ and $d_k = v_k - x_k$. We have

$$\begin{aligned} \|v_k - x_*\| &\leq \|v_k - x_k\| + \|x_k - x_*\| \\ &\leq \omega_k + \|x_k - x_*\|. \end{aligned}$$

Since, by hypothesis, $\omega_k \to 0$ and $x_k \to x_*$ for $k \in \mathcal{K}$, the two terms on the right hand side converge to 0, which implies that we also have

$$\lim_{k \in \mathcal{K}} v_k = x_*.$$

Therefore, there is a threshold index k_0 such that all bounds inactive at x_* are also inactive at v_k for $k \geq k_0$. This is equivalent to $I(v_k) \subseteq I_*$ for $k \geq k_0$. We then have the reverse inclusion for the associated tangent spaces $T(x_*) \subseteq T(v_k)$, giving

$$\left\|P_{T(x_*)}\left[\nabla_x \Phi_k\right]\right\| \leq \left\|P_{T(v_k)}\left[\nabla_x \Phi_k\right]\right\|, \tag{3.25}$$

whenever $k \in \mathcal{K}$ is above the threshold index k_0.

But recall from Section 2 that $T(v_k)$ is the null space of a matrix $\tilde{A}_k$ of the form (2.8). Moreover, since v_k is a projection onto a convex set, it can be characterized by

$$x_k - \nabla_x \Phi_k - v_k = -\left(\nabla_x \Phi_k + d_k\right) \in \mathcal{N}_\Omega(v_k).$$

By the expression (3.24), $\mathcal{N}_\Omega(v_k)$ is trivially a subset of the row space of $\tilde{A}_k$ and the latter is also the orthogonal complement of the null space of $\tilde{A}_k$, i.e. $T(v_k)^\perp$. Because $T(v_k)^\perp$ is a vector space:

$$\nabla_x \Phi_k + d_k \in T(v_k)^\perp,$$

and we obtain, by linearity of $P_{T(v_k)}[\cdot]$:

$$P_{T(v_k)}[\nabla_x \Phi_k + d_k] = 0 \iff P_{T(v_k)}[\nabla_x \Phi_k] = -P_{T(v_k)}[d_k]. \tag{3.26}$$

Combining (3.25) and (3.26) gives

$$\left\|P_{T(x_*)}[\nabla_x \Phi_k]\right\| \leq \left\|P_{T(v_k)}[\nabla_x \Phi_k]\right\| = \left\|P_{T(v_k)}[d_k]\right\| \leq \|d_k\| \leq \omega_k,$$

whenever $k \in \mathcal{K}$ is above the threshold k_0, which is the desired inequality. □

The next lemma is a standard result about the behavior of the Lagrange multipliers in AL methods.

Lemma 3.5 (Lemma 14.4.1 [22]). *Assume that* $\lim_{k\to\infty} \mu_k = \infty$ *when Algorithm 1 is executed. Then*

$$\lim_{k\to\infty} \frac{\|\lambda_k\|}{\mu_k} = 0.$$

In the spirit of [21, Lemma 4.3], we state the main convergence results.

Lemma 3.6. *Suppose that assumptions 1, 3 hold. Let* $(x_k)_{k\in\mathcal{K}}$ *be a sequence of iterates in* Ω *satisfying Assumption 7 and converging to* x_* *for which Assumption 8 holds. Let* $\lambda_* = \lambda(x_*)$, $(\lambda_k)_{k\in\mathcal{K}}$ *be any sequence of vectors and* $(\mu_k)_{k\in\mathcal{K}}$ *be a non-decreasing sequence of positive scalars. Assume that* (2.6) *holds with* $\lim_{k\in\mathcal{K}} \omega_k = 0$.

Then, there are positive constants κ_2, κ_3 *such that*

$$\|\bar{\lambda}_k - \lambda_*\| \leq \kappa_2 \omega_k + \kappa_3 \|x_k - x_*\|, \tag{3.27}$$

$$\|\lambda(x_k) - \lambda_*\| \leq \kappa_3 \|x_k - x_*\|, \tag{3.28}$$

and

$$\|c(x_k)\| \leq \kappa_2 \frac{\omega_k}{\mu_k} + \kappa_3 \frac{\|x_k - x_*\|}{\mu_k} + \frac{\|\lambda_k - \lambda_*\|}{\mu_k}. \tag{3.29}$$

If, in addition, $c(x^*) = 0$, *then* x_* *is a first-order critical point for problem* (1.1) *with corresponding Lagrange multipliers* λ_* *and with* $\lim_{k\in\mathcal{K}} \bar{\lambda}_k = \lim_{k\in\mathcal{K}} \lambda(x_k) = \lambda_*$.

Proof. By assumptions 1 and 8, for k sufficiently large, the matrix $(C_k N_*)^\dagger$ is well defined, bounded and converges to $(C_* N_*)^\dagger$. Therefore, there is a constant κ_2 such that

$$\|((C_k N_*)^\dagger)^\top\| \leq \kappa_2.$$

Furthermore, by Lemma 3.4, we have

$$\|N_*^\top \nabla_x \Phi_k\| = \left\|P_{T(x_*)}[\nabla_x \Phi_k]\right\| \leq \omega_k,$$

for $k \in \mathcal{K}$ large enough. Thus we may deduce

$$\begin{aligned}\|\bar{\lambda}_k - \lambda(x_k)\| &= \|((C_k N_*)^\dagger)^\top N_*^\top \nabla f_k + \bar{\lambda}_k\| \\ &= \|((C_k N_*)^\dagger)^\top (N_*^\top \nabla f_k + (C_k N_*)^\top \bar{\lambda}_k)\| \\ &= \|((C_k N_*)^\dagger)^\top (N_*^\top \nabla_x \Phi_k)\| \\ &\leq \|((C(x) N_*)^\dagger)^\top\| \, \|N_*^\top \nabla_x \Phi_k\| \\ &\leq \kappa_2 \omega_k.\end{aligned} \tag{3.30}$$

Furthermore, by the mean value theorem for vector-valued functions, we have

$$\lambda(x_k) - \lambda_* = \int_0^1 \nabla\lambda(x(t))(x_k - x_*)dt,$$

where $x(t) = x_k + t(x_* - x_k)$. The integrand is bounded for x_k sufficiently close to x_*, hence

$$\|\lambda(x_k) - \lambda_*\| \leq \kappa_3 \|x_k - x_*\|,$$

for $k \in \mathcal{K}$ sufficiently large and for some positive constant κ_3, giving (3.28). Then, combining the latter inequality and (3.30), we may deduce that

$$\begin{aligned}\|\bar{\lambda}_k - \lambda_*\| &\leq \|\bar{\lambda}_k - \lambda(x_k)\| + \|\lambda(x_k) - \lambda_*\| \\ &\leq \kappa_2 \omega_k + \kappa_3 \|x_k - x_*\|,\end{aligned}$$

which is inequality (3.27). Next, by expression (2.2), we have $c(x_k) = (\bar{\lambda}_k - \lambda_k)/\mu_k$, which leads to

$$\begin{aligned}\|c(x_k)\| &\leq \frac{\|\bar{\lambda}_k - \lambda_*\|}{\mu_k} + \frac{\|\lambda_k - \lambda_*\|}{\mu_k} \\ &\leq \kappa_2 \frac{\omega_k}{\mu_k} + \kappa_3 \frac{\|x_k - x_*\|}{\mu_k} + \frac{\|\lambda_k - \lambda_*\|}{\mu_k},\end{aligned}$$

where we have used (3.27). This concludes the first part of the lemma.

Since $(\omega_k)_k$ tends to 0 and $(x_k)_{k\in\mathcal{K}}$ converges to x_* by assumption, inequalities (3.27) and (3.28) imply that both sequences $(\bar{\lambda}_k)_k$ and $(\lambda(x_k))_k$ converge to λ_* for $k \in \mathcal{K}$. Therefore, from identity (2.3), we obtain that the sequence of gradients $\nabla_x \Phi_k$ converge to $\nabla_x \mathcal{L}(x_*, \lambda_*)$.

Finally, assume that $c(x^*) = 0$. Because (2.6) holds, we have on the one hand that

$$\lim_{k\in\mathcal{K}} \|P_\Omega\left[x_k - \nabla_x \Phi_k\right] - x_k\| = 0. \tag{3.31}$$

On the other hand, the set Ω being closed and convex, the projection mapping $P_\Omega[\cdot]$ is continuous, so $P_\Omega\left[x_k - \nabla_x \Phi_k\right] \to P_\Omega\left[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)\right]$ as $k \in \mathcal{K}$ increases. With (3.31), we obtain

$$P_\Omega\left[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)\right] = x_*.$$

Therefore, we have established that (x_*, λ_*) is a KKT point and the lemma is proved. □

We can now establish the global convergence of Algorithm 1.

Theorem 3.3. *Suppose that assumptions 1, 3 hold. Let $(x_k)_k$ be a sequence of iterates generated by Algorithm 1 satisfying Assumption 7. Further assume that there is a subsequence $(x_k)_{k\in\mathcal{K}}$ converging to x_*, for which Assumption 8 holds and let $\lambda_* = \lambda(x_*)$. Then, conclusions of Lemma 3.6 hold.*

Proof. As in the proof of Lemma (3.6), the inequalities (3.27), (3.28), (3.29) can be derived from the assumptions. All that remains is to show that $c(x^*) = 0$. There are two cases to distinguish.

First, if the sequence of penalty parameters is bounded above, then there is k sufficiently large such that $\|c(x_k)\| \leq \eta_k$ for all k. Since $\eta_k \to 0$, we get that $c(x_*) = 0$.

Next, if the sequence of penalty parameters is unbounded, then by the update rule, we have that $(\mu_k)_k$ grows to ∞. Hence, by Lemma 3.5, the ratio λ_k/μ_k converges to zero. Therefore, by inequality (3.29), we also obtain that $c(x_*) = 0$. In both cases, x_* is feasible with respect to the nonlinear constraints and the remaining conclusions of Lemma 3.6 hold. □

Our convergence analysis is based on the criticality measure (2.6) but, as we already mentioned in Section 2, the latter is not available in closed form for a general polyhedron of the form (1.2). This is what justified our choice to monitor criticality in our algorithm through the reduced gradient norm $\|P_{T(x_k)}[\nabla_x \Phi_k]\|$, which only involves the projection onto the tangent space of the currently active bounds. The two measures are linked through the tangent cone $\mathcal{T}_\Omega(x_k)$, which is the dual cone of $\mathcal{N}_\Omega(x_k)$. Indeed, one always has

$$\|P_\Omega\left[x_k - \nabla_x \Phi_k\right] - x_k\| \leq \|P_{\mathcal{T}_\Omega(x_k)}[-\nabla_x \Phi_k]\|$$

and whenever the multipliers associated with the bounds active at x_k are of the correct sign the projection onto the cone reduces to the projection onto $T(x_k)$, so that (see [21, Section 7.2])

$$\|P_{\mathcal{T}_\Omega(x_k)}[-\nabla_x \Phi_k]\| = \|P_{T(x_k)}[\nabla_x \Phi_k]\|.$$

Consequently, under the assumption that these sign conditions hold at every outer iterate, the reduced-gradient rule (2.11) certifies the criticality test (2.6), and Theorem 3.3 applies to the iterates the implementation produces. Without this assumption the reduced gradient controls only the tangential component of $\nabla_x \Phi_*$, and the limit point may not be critical.

4 Numerical experiments

In this section, we report numerical experiments conducted with the Julia implementation of our algorithm, named TRAULLS[1] on a Mac mini with an M4 processor and 24 GB of RAM. Our solver is open-source and can be downloaded from `https://github.com/UncertainLab/Traulls.jl`. All the benchmarks we present here are also accessible in the `benchmark` folder of the linked GitHub repository. The tests were carried out on a set of 49 constrained NLS from collections [35, 41, 55] and problems 2 and 3 referenced in [12]. For problems with variable dimensions, we set the number of variables to $n \in \{100, 500, 1000\}$. Thus, we have a total of 79 problem instances ranging from 2 to 1000 variables, with up to 2500 residuals and 1000 constraints. Problems with inequality constraints were converted to the form (1.1) by adding slack variables. For our tests, we set the criticality tolerance to $\omega_* = 10^{-5}$, the feasibility tolerance to $\eta_* = 10^{-6}$, the maximum number of outer iterations to 500 and the maximum

[1] Trust Region AUgmented nonLinear Least-squares Solver

number of iterations to 1000. We leave the other parameters of the algorithm at the values given at the end of Section 2. We present our results with performance profiles of type Dolan and Moré [27] produced with the `BenchmarkProfiles.jl` package [49].

4.1 Comparison of Hessian approximations

Our first set of experiments compares the behavior and performance of our algorithm depending on the Hessian approximation used during the inner minimization. We compared the GN approximation (2.23) against structured approximations of the form (2.24). Based on the structured secant equation (2.27), we implemented the SR1 update we discussed and the BFGS update that one can derive similarly. Both structured approximations were used in their plain and hybrid variants with the switching rule (2.30). We compared these five strategies against three metrics: computation time, number of outer iterations and number of inner iterations. Results are indicated in Figure 2. All five variants successfully solved all but one problem:

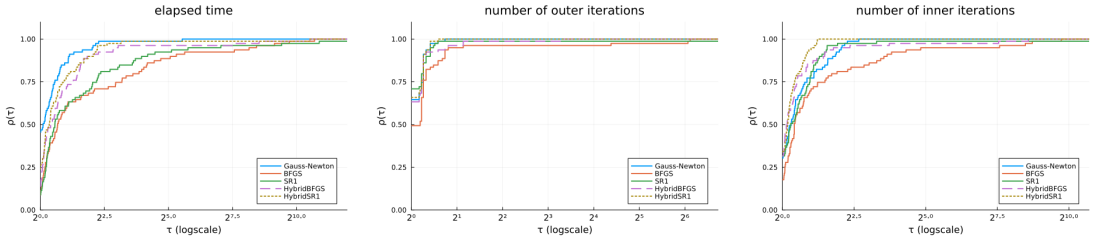

Figure 2: Performance profiles comparing five different Hessian approximations strategies with respect to computation time (left), number of outer iterations (center), and number of inner iterations (right).

the plain SR1 variant reaches the maximum number of iterations on the HS373 problem. In terms of computation time, the GN approximation is the fastest: since it requires no additional storage or update computation, it is the cheapest per iteration as our method mainly involves matrix-vector products. The GN approximation especially dominates on problems having sparse jacobians and small residuals at the solution, since the second order terms are still stored as a dense matrix. When measured by the number of inner iterations, the hybrid variants prove more efficient than their plain counterparts for both formulas. These observations are consistent with the discussion about the hybrid switching rule (2.30) which – as in the unconstrained case – correctly identifies situations where the Gauss–Newton approximation must be corrected and activates the structured quasi-Newton correction accordingly. All approximation approaches are closer in terms of outer iterations, with plain BFGS a little bit behind, but overall, the Hybrid SR1 appears to be the most robust. To confirm these observations, we have focused our attention to a comparison of GN with Hybrid approximations, the latter appearing to be the most efficient. Profiles in Figure 3 compare the 3 approaches against the number of residual and gradient evaluations. The latter are similar to the inner iterations profile previously shown, as we evaluate the residuals one time per inner iteration and evaluate the gradient at every accepted step. Nevertheless, Hybrid-SR1 exhibits the best efficiency and robustness. This also seems to indicate that the SR1 approximation is adequate with the negative curvature handling in the trust region setting. Based on these results, we retain the Hybrid SR1 variant as the best overall strategy for TRAULLS, and use it in the

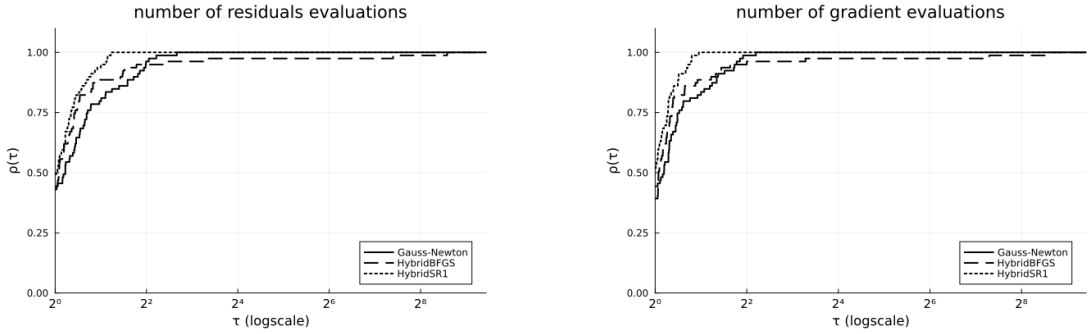

Figure 3: Performance profiles comparing the Gauss-Newton approximation against hybrid approaches with respect to the number of residual evaluations (left), and gradient evaluations (right).

comparisons that follow.

4.2 Comparison with IPOPT and Percival

We now compare TRAULLS (with the Hybrid SR1 approximation) against solvers IPOPT [62] and Percival [3]. The latter is also an augmented Lagrangian algorithm and has a native Julia implementation and uses a Julia version of the TRON [40] algorithm – part of the `JSOSolvers.jl` package [45] – for the inner minimization. To benefit from the benchmarking facilities provided by the JuliaSmoothOptimizers (JSO)[2] ecosystem, we used the wrapper `NLPModelsIpopt.jl` [58]. This allows us to run both solvers on our test set of problems – all models being encoded in `NLSProblems.jl` [56] – and export the benchmark metrics using `SolverBenchmark.jl` [57]. To make the comparison with IPOPT relevant, we have set the parameters `nlp_scaling_method` = `none`, `dual_inf_tol` = `Inf`, `constr_viol_tol` = `Inf`, `compl_inf_tol` = `Inf`, `acceptable_iter` = `0` and the maximum number of iterations to 1000. For Percival, since it also follows Algorithm 1, we have set its tolerances and parameters to the same value as ours. For these tests, we compare the three solvers against the CPU time, the number of residual evaluations and the number of gradient evaluations. The results are presented in Figure 4. During our experiments, IPOPT reached the maximum number of iterations on problem HS27 and returned the `unknown` internal message on problem HS70. Percival reached the maximum of iterations on the two instances of problem 5.11 from [41] having $n = 500$ and $n = 1000$.

In terms of computation time, TRAULLS and IPOPT achieve comparable performance, with IPOPT holding a modest advantage on the fastest problems but the gap narrowing as problem difficulty increases. Percival is consistently slower than both solvers across the test set. Regarding the function-evaluation metrics, IPOPT dominates: its interior-point strategy, which exploits second-order information through exact Hessians, requires significantly fewer residual and gradient evaluations to reach convergence. TRAULLS, which relies on a structured quasi-Newton approximation and therefore does not require exact second derivatives, performs comparably to Percival on these metrics and outperforms it on a majority of problems. Across all three metrics, IPOPT is a clear winner, in spite of the two failures. However, our method

[2] https://jso.dev/

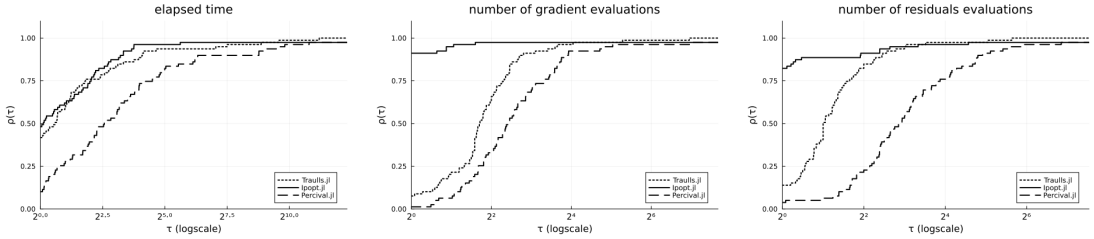

Figure 4: Performance profiles comparing TRAULLS (Hybrid SR1), IPOPT, and Percival on the test set, with respect to computation time (left), residual evaluations (center), and gradient evaluations (right).

is a close second in computation time and achieves similar robustness with regard to the number of residual evaluations. The clear advantage of TRAULLS over Percival shows that within the same algorithmic framework, exploiting the least-squares structures leads to better performance.

Conclusion

In this paper, we have presented an algorithm able to solve NLS problems subject to constraints of general form. Our method fully exploits the structure of the problem through the AL reformulation and a structured Hessian approximation. Although our implementation exhibits promising performance, its limitation for large-scale applications lies in the Hessian approximation: the dense storage of the structured second order correction and the associated computations become prohibitive as the number of variables grows. A limited-memory variant of the structured update, following [14], would be a natural extension to address this bottleneck. To tackle problems with a large number of residuals, another avenue of research could be to combine the AL framework with an adaptive sampling strategy applied to the residuals, lowering the dimensions of the inner minimization subproblems. This would extend to the constrained case techniques currently investigated for unconstrained problems [8].

Acknowledgments: The authors would like to thank Natural Sciences and Engineering Research Council of Canada (NSERC) and the Institute for Data Valorization (IVADO) in addition to Mitacs and Hydro-Québec for their financial support through grants IT40867 and IT38322.

Appendix

A Block Cholesky factorization of the augmented Gram matrix

Let A be an $m \times n$ full row rank matrix for which the Cholesky factor of the Gram matrix $AA^\top$ is available. Considering $\mathcal{A} = \{i_1, \ldots, i_p\} \subseteq \{1, \ldots, n\}$, let $Z \in \mathbb{R}^{p \times n}$ be the matrix whose row k is the row i_k of the $n \times n$ identity. In this appendix, we show how to construct the Cholesky decomposition of the Gram matrix $MM^\top$, where M is defined as

$$M = \begin{bmatrix} A \\ Z \end{bmatrix} \in \mathbb{R}^{(m+p)\times n}.$$

Following the reasoning in [30, section 4.2.9] for the Cholesky decomposition of a general block matrix, we will show how, at the cost of extra computations, we can recover the Cholesky factor of $MM^\top$ using its block structure and the factor L. The definition of M naturally leads to the block pattern

$$MM^\top = \begin{bmatrix} AA^\top & AZ^\top \\ ZA^\top & ZZ^\top \end{bmatrix}$$

Simple computations first show that

$$ZZ^\top = I_p, \quad AZ^\top = [A_{i_1}, \ldots, A_{i_p}].$$

The latter, written $A_{|\mathcal{A}}$, is the restriction of A to the columns corresponding to the indices in $\mathcal{A}$. We can write $MM^\top$ as

$$\begin{bmatrix} AA^\top & A_{|\mathcal{A}} \\ A_{|\mathcal{A}}^\top & I_p \end{bmatrix} \tag{A.1}$$

Let $\tilde{L}$ be $(m+p) \times (m+p)$ triangular such that $MM^\top = \tilde{L}\tilde{L}^\top$. We apply the same block structure to $\tilde{L}$, i.e.

$$\tilde{L} = \begin{bmatrix} \tilde{L}_{11} & 0 \\ G_{21} & \tilde{L}_{22}, \end{bmatrix},$$

with

- $\tilde{L}_{11}$ $m \times m$ lower triangular
- G_{21} $p \times m$
- $\tilde{L}_{22}$ $p \times p$ lower triangular

Verifying $MM^\top = \tilde{L}\tilde{L}^\top$ implies the matrix equality

$$\begin{bmatrix} AA^\top & A_{|\mathcal{A}} \\ A_{|\mathcal{A}}^\top & I_p \end{bmatrix} = \begin{bmatrix} \tilde{L}_{11}\tilde{L}_{11}^\top & \tilde{L}_{11}G_{21}^\top \\ G_{21}\tilde{L}_{11}^\top & G_{21}G_{21}^\top + \tilde{L}_{22}\tilde{L}_{22}^\top \end{bmatrix}. \tag{A.2}$$

By comparison of the blocks in (A.2), it follows that

$$\begin{aligned} AA^\top &= \tilde{L}_{11}\tilde{L}_{11}^\top \\ A_{|\mathcal{A}} &= \tilde{L}_{11}G_{21}^\top \\ I_p &= G_{21}G_{21}^\top + \tilde{L}_{22}\tilde{L}_{22}^\top. \end{aligned}$$

Therefore, we can form $\tilde{L}$ by first setting $\tilde{L}_{11} = L$, then solve p lower triangular systems to compute G_{21} and finally compute the Cholesky factor of $I_p - G_{21}G_{21}^\top$ to get $\tilde{L}_{22}$. We briefly justify why the latter is well defined. By definition of $G_{21}^\top$ and the full rank assumption of A, one has

$$G_{21}^\top = \left(\tilde{L}_{11}\right)^{-1} A_{|\mathcal{A}} = L^{-1}A_{|\mathcal{A}}.$$

Then, we obtain

$$\begin{aligned} I_p - G_{21}G_{21}^\top &= I_p - \left(L^{-1}A_{|\mathcal{A}}\right)^\top \left(L^{-1}A_{|\mathcal{A}}\right) \\ &= I_p - A_{|\mathcal{A}}^\top \left(LL^\top\right)^{-1} A_{|\mathcal{A}} \\ &= I_p - A_{|\mathcal{A}}^\top \left(AA^\top\right)^{-1} A_{|\mathcal{A}}, \end{aligned}$$

which is nothing but the Schur complement of $AA^\top$ of the augmented matrix (A.1). Thus, the positive definiteness of $I_p - G_{21}G_{21}^\top$ comes from the positive definiteness of $MM^\top$ (also implied by the full rank assumption of A).

B Cauchy point computation

We now describe a procedure to compute the Cauchy point as the first local minimizer of the quadratic model along the projected gradient path adapted from [15] to the case where linear equalities are present. For the sake of clarity, we omit the iteration index during the rest of this subsection. The piece-wise arc $s(t) = P_\Omega\left[x - tg\right] - x$ satisfies the constraints

- $As(t) = 0$
- $s^{(l)} \leq s(t) \leq s^{(u)}$,

with $s_i^{(l)} = \max(-\Delta, l_i - x_i)$ and $s_i^{(u)} = \min(\Delta, u_i - x_i)$.

The following breakpoints:

$$0 = t_0 < t_1 < \ldots < t_p,$$

correspond to the successive scalars at which at least one component of $s(t)$ satisfies a bound with equality. Note that since A has m rows and is full rank, there are $n-m$ degrees of freedom remaining and thus at most $n-m$ breakpoints before the projected gradient path is constant. We recall the recursive expression

$$s(t) = -(t - t_i)z_i + s(t_i),$$

for $t \in [t_i, t_{i+1})$ and with the reduced gradient z_i given by (3.5).

To find the first local minimum of the scalar function $\psi(t) = q(s(t))$, we successively study each interval $[t_i, t_{i+1})$ to assert whether or not it contains a local minimizer. Assume we have not found a local minimizer on $[t_0, t_i)$ and thus look at the interval $[t_i, t_{i+1})$. The model along this arc can be written

$$\psi(t) = \frac{\psi_i''}{2}(\Delta t)^2 + \psi_i'\Delta t \tag{B.1}$$

with

- $\psi_i'' = \langle z_i, Hz_i\rangle$
- $\psi_i' = -\langle s(t_i), Hz_i\rangle - \langle g, z_i\rangle$

- $\Delta t = t - t_i$

Different cases can occur depending on the values of the slope ψ_i' and the curvature ψ_i''. Firstly, if

$$\begin{aligned} &\psi_i' > 0 \text{ or} \\ &\psi_i' = 0 \text{ and } \psi_i'' > 0, \end{aligned}$$

then t_i is the required minimizer. Next, if

$$\psi_i' < 0 \text{ and } \psi_i'' > 0,$$

the quadratic (B.1) has a strict minimizer at

$$t_i - \frac{\psi_i'}{\psi_i''}.$$

If the latter belongs to the interval of interest, i.e.

$$t_i - \psi_i'/\psi_i'' < t_{i+1},$$

then it is the required minimizer. In other cases, the minimizer is at or beyond t_{i+1}. To prepare for the study of the next interval, we first need to find the next breakpoint, given by the smallest scalar $t_{i+1} > t_i$ such that, at $s(t_{i+1})$, one of the free components hits one of its bounds. By introducing

$$\delta_i^- = \min_{(z_i)_j > 0} \left\{ (s(t_i)_j - s_j^{(l)})/(z_i)_j \right\}, \qquad \delta_i^+ = \min_{(z_i)_j < 0} \left\{ (s(t_i)_j - s_j^{(u)})/(z_i)_j \right\},$$

the next breakpoint is given by

$$t_{i+1} = t_i + \delta_i, \tag{B.2}$$

with $\delta_i = \min\left(\delta_i^-, \delta_i^+\right)$. We then add the corresponding variable index to the list of fixed components, compute the next reduced gradient z_{i+1} and finally update the slope and curvature of the model along the next interval. A last termination case can occur. Since A is of rank m, at most $n - m$ bounds can become active so if we never find a local minimum, the procedure still ends whenever it reaches the last breakpoint t_p. Indeed, past this breakpoint, the model along the projected gradient path is constant so we can return the last accumulated step $s(t_p)$ as the Cauchy step. Note that, in this case, it is also the total step, because no more variables can be modified. The presence of the trust region constraint ensures that all bounds become active. The procedure for the Cauchy step computation is outlined in Algorithm 4.

Algorithm 4 Cauchy step computation

Require: Hessian H, gradient g, bounds on the direction $s^{(l)},\ s^{(u)}$

Identify $I(0)$ and compute the direction z_0
Set **found** $\leftarrow$ **false**
Set $t_0 \leftarrow 0$, $s^{(0)} \leftarrow 0$ and initialize counter $i \leftarrow 0$
repeat
    $\psi_i' \leftarrow -\left\langle s^{(i)}, Hz_i \right\rangle - \langle g, z_i \rangle\ ,\ \psi_i'' \leftarrow \langle z_i, Hz_i \rangle$
    Find the next breakpoint t_{i+1} by (B.2)
    if $\psi_i' > 0$ or $\psi_i' = 0$ and $\psi_i'' > 0$ **then**
        Set $s^C \leftarrow s^{(i)}$
        found $\leftarrow$ **true**
    else if $\psi_i' < 0$ and $\psi_i'' > 0$ and $t_i - \psi_i'/\psi_i'' < t_{i+1}$ **then**
        Set $\Delta t \leftarrow -\psi_i'/\psi_i''$
        Set$s^C \leftarrow s^{(i)} - \Delta t z_i$, **found** $\leftarrow$ **true**
    else
        Set $\Delta t \leftarrow t_{i+1} - t_i$
        Compute the next direction z_{i+1}
        Set $s^{(i+1)} \leftarrow s^{(i)} - \Delta t z_i$
    end if
    Increment $i \leftarrow i + 1$
    if $|I(t_i)| = n - m$ **then**
        Set $s^C \leftarrow s^{(i)}$, **found** $\leftarrow$ **true**
    end if
until found
return s^C

References

[1] M. Al-Baali and R. Fletcher. Variational methods for non-linear least-squares. *The Journal of the Operational Research Society*, 36(5):405–421, 1985. doi: 10.1057/jors.1985.68.

[2] R. Andreani, E.G. Birgin, J.M. Martinez, and M.L. Schuverdt. On Augmented Lagrangian methods with general lower-level constraints. *SIAM Journal on Optimization*, 18(4):1286–1309, 2008. doi: 10.1137/060654797.

[3] S. Arreckx, A. Lambe, J.R.R.A. Martins, and D. Orban. A matrix-free augmented Lagrangian algorithm with application to large-scale structural design optimization. *Optimization and Engineering*, 17:359–384, 2016. doi: 10.1007/s11081-015-9287-9.

[4] M.C. Bartholomew-Biggs. The estimation of the hessian matrix in nonlinear least-squares problems with non-zero residuals. *Mathematical Programming*, 12:67–80, 1977. doi: 10.1007/BF01593770.

[5] S. Bellavia, M. Macconi, and B. Morini. STRSCNE: A scaled trust-region solver for constrained nonlinear equations. *Computational Optimization and Applications*, 28:31–50, 2004.

[6] S. Bellavia, J. Gondzio, and B. Morini. Computational experience with numerical methods for nonnegative least-squares problems. *Numerical Linear Algebra with Applications*, 18 (3):363–385, 2011.

[7] S. Bellavia, S. Gratton, and E. Riccietti. A Levenberg-Marquardt method for large nonlinear least-squares problems with dynamic accuracy in functions and gradients. *Numerische Mathematik*, 140:791–825, 2018. doi: 10.1007/s00211-018-0977-z.

[8] Stefania Bellavia, Greta Malaspina, and Benedetta Morini. A variable dimension sketching strategy for nonlinear least-squares. *SIAM Journal on Scientific Computing*, 48(3):A1206–A1234, 2026. doi: 10.1137/25M175980X.

[9] E.H. Bergou, Y. Diouane, V. Kungurtsev, and C.W. Royer. A nonmonotone matrix-free algorithm for nonlinear equality-constrained least-squares problems. *SIAM Journal on Scientific Computing*, 43(5):S743–S766, 2021. doi: 10.1137/20M1349138.

[10] J.T. Betts. Solving the nonlinar least square problem: Application of a general method. *Journal of Optimization Theory and Applications*, 18:469–483, 1976. doi: 10.1007/BF00932656.

[11] J. Bezanson, A. Edelman, S. Karpinski, and V.B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi: 10.1137/141000671.

[12] L.T. Biegler, J. Nocedal, C. Schmid, and D. Ternet. Numerical experience with a reduced hessian method for large scale constrained optimization. *Computational Optimization and Applications*, 15:45–67, 2000. doi: 10.1023/A:1008723031056.

[13] Å ke Björck. *Numerical Methods for Least Squares Problems*. SIAM, Philadelphia, PA, USA, 2nd edition, 2024. doi: 10.1137/1.9781611977950.

[14] R.H. Byrd, J. Nocedal, and R.B. Schnabel. Representations of quasi-Newton matrices and their use in limited memory methods. *Mathematical Programming*, 63:129–156, 1994. doi: 10.1007/BF01582063.

[15] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. Global convergence of a class of trust region method algorithms for optimization with simple bounds. *SIAM Journal on Numerical Analysis*, 25(2):433–460, 1988. doi: 10.1137/0725029.

[16] A.R. Conn, N.I.M Gould, and Ph.L. Toint. Testing a class of methods for solving minimization problems with simple bounds on the variables. *Mathematics of Computation*, 50 (182):399–430, 1988. doi: 10.1090/S0025-5718-1988-0929544-3.

[17] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. A globally convergent Augmented Lagrangian algorithm for optimization with general constraints and simple bounds. *SIAM Journal on Numerical Analysis*, 28(2):545–572, 1991. doi: 10.1137/0728030.

[18] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. Convergence of quasi-Newton matrices generated by the symmetric rank one update. *Mathematical Programming*, 50:177–195, 1991. doi: 10.1007/BF01594934.

[19] A.R. Conn, N.I.M Gould, and Ph.L. Toint. *LANCELOT: A Fortran Package for Large-Scale Nonlinear Optimization (Release A)*. Springer Series in Computational Mathematics. Springer Berlin, Heidelberg, 1992. doi: 10.1007/978-3-662-12211-2.

[20] A.R. Conn, N. Gould A. Sartenaer, and Ph.L. Toint. Global convergence of a class of trust region algorithms for optimization using inexact projections on convex constraints. *SIAM Journal on Optimization*, 3(1):164–221, 1993. doi: 10.1137/0803009.

[21] A.R. Conn, N. Gould A. Sartenaer, and Ph.L. Toint. Convergence properties of an Augmented Lagrangian algorithm for optimization with a combination of general equality and linear constraints. *SIAM Journal on Optimization*, 6(3):674–703, 1996. doi: 10.1137/S1052623493251463.

[22] A.R. Conn, N.I.M Gould, and Ph.L. Toint. *Trust Region Methods*. SIAM, Philadelphia, PA, USA, 2000. doi: 10.1137/1.9780898719857.

[23] F.E. Curtis, H. Jiang, and D.P. Robinson. An adaptative Augmented Lagrangian method for large-scale constrained optimization. *Mathematical Programming, Series A*, 152:201–245, 2015. doi: 10.1007/s10107-014-0784-y.

[24] F. Delbos, J.Ch. Gilbert, R.Glowinski, and D. Sinoquet. Constrained optimization in seismic reflection tomography: a Gauss-Newton augmented Lagrangian approach. *Geophysic Journal International*, 164:670–684, 2006. doi: 10.1111/j.1365-246X.2005.02729.x.

[25] J.E. Dennis Jr and R.B. Schnabel. *Numerical Methods for Unconstrained optimization and Nonlinear Equations*. Classics in Applied Mathematics. SIAM, Philadelphia, PA, USA, 1996. doi: 10.1137/1.9781611971200.

[26] J.E. Dennis Jr, D.M. Gay, and R.E. Walsh. An adaptive nonlinear least-squares algorithm. *ACM Transactions on Mathematical Software*, 7(3):348–368, 1981. doi: 10.1145/355958.355965.

[27] E.D. Dolan and J.J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, Series A 91(2):201–213, 2002. doi: 10.1007/s101070100263.

[28] R. Fletcher and C. Xu. Hybrid methods for nonlinear least squares. *IMA Journal of Numerical Analysis*, 7:373–389, 1987. doi: 10.1093/imanum/7.3.371.

[29] P.E. Gill and D.P. Robinson. A primal-dual Augmented Lagrangian. *Computationnal Optimization and Applications*, 51:1–25, 2012. doi: 10.1007/s10589-010-9339-1.

[30] G.H. Golub and C.F. Van Loan. *Matrix Computations*. JHU Press, Maryland, MD, USA, 4th edition, 2013. doi: 10.56021/9781421407944.

[31] M.L.N. Gonçalves and T.C. Menezes. Gauss–Newton methods with approximate projections for solving constrained nonlinear least squares problems. *Journal of Complexity*, 58: 101459, 2020. doi: 10.1016/j.jco.2020.101459.

[32] N.I.M. Gould, M.E. Hribar, and J. Nocedal. On the solution of equality constrained quadratic programming problems arising in optimization. *SIAM Journal on Scientific Computing*, 23(4):1376–1395, 2001. doi: 10.1137/S1064827598345667.

[33] Michel Grenier, J. Gagnon, C. Mercier, and J. Richard. Short-term load forecasting at Hydro-Québec TransÉnergie. In *2006 IEEE Power Engineering Society General Meeting*, Montreal, QC, Canada, 2006. doi: 10.1109/PES.2006.1709029.

[34] M.R. Hestenes. Multipler and gradient methods. *Journal of Optimization Theory and Applications*, 4:303–320, 1969. doi: 10.1007/BF00927673.

[35] W. Hock and K. Schittkowski. *Test Examples for Nonlinear Programming Codes*, volume 187 of *Lecture Notes in Economics and Mathematical Systems*. Springer Berlin, Heidelberg, 2nd edition, 1980. doi: 10.1007/978-3-642-48320-2.

[36] J. Huschens. On the use of product structure in secant methods for nonlinear least squares problems. *SIAM Journal on Optimization*, 4(1):108–129, 1994. doi: 10.1137/0804005.

[37] J.E.Dennis Jr, H.J. Martinez, and R.A. Tapia. Convergence theory for the structured BFGS method with an application to nonlinear least squares. *Journal of Optimization Theory and Applications*, 61(2):161–178, 1999. doi: 10.1007/BF00962795.

[38] K. Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944. doi: 10.1090/qam/10666.

[39] Z. Li, M. Osborne, and T. Prvan. Adaptative algorithm for constrained least-squares problems. *Journal of Optimization Theory and Applications*, 114:423–441, 2002. doi: 10.1023/A:1016043919978.

[40] C.-J. Lin and J.J Moré. Newton's method for large bound-constrained optimization problems. *SIAM Journal on Optimization*, 9(4):1100–1127, 1999. doi: 10.1137/S1052623498345075.

[41] L. Lukšan and J. Vlček. Sparse and partially separable test problems for unconstrained and equality constrained optimization. Technical Report 767, Institute of Computer Science, Academy of Sciences of the Czech Republic, 1999.

[42] L. Lukšan, C. Matonoha, and J. Vlček. Hybrid methods for nonlinear least squares problems. Technical Report 1246, Institute of Computer Science, Academy of Sciences of the Czech Republic, 2019.

[43] N. Mahdavi-Amiri and R.H. Bartels. Constrained nonlinear least squares: An exact penalty approach with projected structured quasi-Newton update. *ACM Transcations on Mathematical Software*, 15(3):220–242, 1989. doi: 10.1145/66888.66891.

[44] D.W. Marquardt. An algorithm for least squares estimation of non-linear parameters. *SIAM Journal*, 11:431–441, 1963. doi: 10.1137/0111030.

[45] T. Migot, D. Monnet, D. Orban, and S. Abel Soares. JSOSolvers.jl: Unconstrained and bound-constrained optimization solvers. *Journal of Open Source Software*, 11(117):9467, 2026. doi: 10.21105/joss.09467. URL `https://doi.org/10.21105/joss.09467`.

[46] J.J. Moré and G. Toraldo. On the solution of large quadratic programming problems with bound constraints. *SIAM Journal on Optimization*, 1(1):93–113, 1991. doi: 10.1137/0801008.

[47] B.A. Murtagh and M.A. Saunders. Large-scale linearly constrained optimization. *Mathematical Programming*, 14:41–72, 1978. doi: 10.1007/BF01588950.

[48] J. Nocedal and S.J. Wright. *Numerical Optimization*. Springer series in Operation Reasearch and Financial Engineering. Springer, New York, NY, USA, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

[49] D. Orban. BenchmarkProfiles.jl, 2019. URL `https://doi.org/10.5281/zenodo.4630955`.

[50] D. Orban and A.S. Siqueira. A regularization method for constrained nonlinear least squares. *Computational Optimization and Applications*, 76(3):961–989, 2020. doi: 10.1007/s10589-020-00201-2.

[51] M. Porcelli. On the convergence of an inexact Gauss-Newton trust-region method for nonlinear least-squares problems with simple bounds. *Optimization Letters*, 7:447–465, 2013. doi: 10.1007/s11590-011-0430-z.

[52] M.J.D. Powell. A method for nonlinear constraints in minimization problems. In R. Fletcher, editor, *Optimization*, pages 283–298. Academic Press, London, 1969.

[53] H. Pu, Z. Chen, B. Wang, and W. Xia. Constrained least squares algorithms for nonlinear unmixing of hyperspectral imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 53(3):1287–1303, 2015. doi: 10.1109/TGRS.2014.2336858.

[54] R.T. Rockafellar. The multiplier method of Hestenes and Powell applied to convex programming. *Journal of Optimization Theory and Applications*, 12(6):555–562, 1973. doi: 10.1007/BF00934777.

[55] K. Schittkowski. *More Test Examples for Nonlinear Programming Codes*. Lecture Notes in Economics and Mathematical Systems. Springer Berlin, Heidelberg, 1987. doi: 10.1007/978-3-642-61582-5.

[56] A. Soares Siqueira and D. Orban. NLSProblems.jl, 2019. URL `https://doi.org/10.5281/zenodo.4605405`.

[57] A. Soares Siqueira and D. Orban. SolverBenchmark.jl, 2020. URL `https://doi.org/10.5281/zenodo.3948381`.

[58] A. Soares Siqueira and D. Orban. NLPModelsIpopt.jl, 2026. URL `https://doi.org/10.5281/zenodo.19420624`.

[59] R. Tapia. On secant updates for use in general constrained optimization. *Mathematics of Compuations*, 51(183):181–202, 1988. doi: 10.2307/2008585.

[60] I.B. Tjoa and L.T. Biegler. Simultaneous solution and optimization strategies for parameter estimation of differential-algebraic equation systems. *Industrial & Engineering Chemistry Research*, 30(2):376–385, 1991. doi: 10.1021/ie00050a015.

[61] Ph.L. Toint and D. Tuyttens. LSNNO, a FORTRAN subroutine for solving large-scale nonlinear network optimization problems. *ACM Transactions on Mathematical Software*, 18(3):308–328, 1992. doi: 10.1145/131766.131771.

[62] A. Wächter and L.T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming, Series A*, 106:25–57, 2006. doi: 10.1007/s10107-004-0559-y.

[63] H. Yabe and T. Takahashi. Factorized quasi-Newton methods for nonlinear least squares problems. *Mathematical Programming*, 51:75–100, 1991. doi: 10.1007/BF01586927.

[64] J.Z. Zhang, L.H. Chen, and N.Y. Deng. A family of scaled factorized broydon-like methods for nonlinear least squares problems. *SIAM Journal on Optimization*, 10(4):1163–1179, 2000. doi: 10.1137/S105262349834530.

[65] W. Zhou and X. Chen. Global convergence of a new hybrid Gauss-Newton structured BFGS method for nonlinear least squares problems. *SIAM Journal on Optimization*, 20 (5):2422–2441, 2010. doi: 10.1137/090748470.